Year: 2026-2027 Sem V-NEP
Course: Agile Software Development & DevOps Lab
Class: T1 Date of Submission: [DD/MM/2026]

A Mini Project Report on

CivicFix

A Smart Civic Complaint Management System Delivered Through an Agile and DevOps Workflow

Submitted in partial fulfilment of the requirements of the course
Agile Software Development & DevOps Lab
Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Rishi Manuja	2401059
Khushaan Matlani	2401063
Hiten Pahilajani	2401069

Under the guidance of
Prof. Sanober Shaikh
Department of Artificial Intelligence & Data Science
Thadomal Shahani Engineering College, 2026–2027

CERTIFICATE

_____ _____

Project Guide Head of Department

Date: _____

Place: _____

ABSTRACT

CivicFix is a Smart Civic Complaint Management System developed to demonstrate how the concepts learned in the ASD & DevOps laboratory can be integrated into a single working software project. It replaces informal ways of reporting civic problems — phone calls, paper applications and word of mouth — with a transparent, trackable web platform. Citizens can report issues such as potholes, overflowing garbage bins, blocked drains and broken streetlights by submitting a description, location and photograph, and then follow the complaint using a unique reference number. Municipal staff verify complaints, assign them to departments, update their status and record resolutions, while administrators manage users, categories and departments. The application was developed using Jinja2 templates, HTML and CSS for the frontend, Python Flask with SQLAlchemy for the backend, and PostgreSQL as the database in the containerized environment.

The project progressively applies the seven laboratory experiments. Git and GitHub provide source-code and version management, including feature branches, a pull request and a rollback using git revert; Docker packages the Flask application into a portable multi-stage image; Docker Compose runs the application, PostgreSQL, Prometheus, node-exporter and Grafana as interconnected services; Jenkins provides Continuous Integration and is extended to automate Docker image building, deployment and a smoke test; Jira demonstrates an Agile Scrum workflow with epics, stories, sprints and dependencies; and Prometheus with Grafana provides application and server monitoring through the /metrics endpoint. This produces a unified software-development and DevOps workflow rather than isolated demonstrations of individual tools.

Keywords: Agile, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Jira, Prometheus, Grafana, Flask, PostgreSQL, Civic Tech

CONTENTS

Chapter 1 Introduction

Chapter 2 Requirements and Agile Planning (Experiment 6)

Chapter 3 System Design

Chapter 4 Application Implementation

Chapter 5 DevOps Implementation (Experiments 1–7)

Chapter 6 Testing

Chapter 7 Implementation (Application Screenshots)

Chapter 8 Results and Discussion

Chapter 9 Conclusion and Future Scope

Appendix A Running the Project

1. Introduction

1.1 Background

Civic problems such as potholes, uncollected garbage, blocked drains and non-working streetlights are reported to municipal bodies through phone calls, written applications or visits to ward offices. Citizens rarely get an acknowledgement, cannot tell who is handling their complaint, and have no way to know whether it was resolved; the municipal office, in turn, struggles to see which complaints are pending, which department holds them and which have crossed a reasonable time limit. CivicFix moves this process online. It provides a practical context in which the concepts taught through the laboratory experiments can be applied progressively to one application.

The laboratory covers version control, containerization, multi-service deployment, continuous integration, automated deployment, Agile project management and continuous monitoring. CivicFix brings these practices together so that development, deployment, planning and monitoring form one connected lifecycle.

1.2 Problem Statement

To design and develop a web-based Smart Civic Complaint Management System and deliver it through an integrated Agile and DevOps workflow in which source code is version-controlled, the application is containerized, dependent services are orchestrated, changes are automatically tested and deployed, development tasks are tracked through Jira, and the running application and server are monitored.

1.3 Objectives

1. Develop a civic complaint management application with citizen, municipal-staff and administrator functionality.
2. Allow citizens to register, log in, report issues with description, location and photo, receive a unique reference number and track the progress of their complaints.
3. Allow municipal staff to verify or reject complaints, assign them to departments, update their status, record resolutions, see overdue complaints on a dashboard and export the complaint register.
4. Allow administrators to manage users, roles, departments and complaint categories.
5. Use Git and GitHub for source-code version control, branching, commits, pull requests, merging and rollback.
6. Containerize the Flask application using Docker and run the application, PostgreSQL and the monitoring stack as interconnected services using Docker Compose.
7. Configure Jenkins for Continuous Integration and extend the pipeline for automated Docker image building, deployment and smoke testing.

8. Use Jira to represent an Agile Scrum workflow with epics, stories, tasks, sprints and dependencies.
9. Integrate Prometheus and Grafana to collect and visualise application and server (CPU, memory, disk) metrics.

1.4 Scope

The project scope is the CivicFix web application and the DevOps workflow demonstrated through Experiments 1 to 7. The report focuses on the citizen, municipal-staff and administrator functions and on integrating Git, GitHub, Docker, Docker Compose, Jenkins, Jira, Prometheus and Grafana.

1.5 Mapping of Lab Experiments to the Project

Exp.	Lab Topic	Application in CivicFix
1	Version Control using Git and GitHub	Repository, commits, feature branches, pull request, merging and rollback with git revert
2	Containerization using Docker	Multi-stage Dockerfile (test and production), image, container and port mapping
3	Multi-Service Deployment using Docker Compose	Flask + PostgreSQL + Prometheus + node-exporter + Grafana, volumes, health check and service networking
4	Continuous Integration using Jenkins	Jenkins pipeline from the Jenkinsfile, pytest with JUnit report, GitHub SCM polling
5	Automated Deployment using GitHub and Jenkins	Docker image build, Docker Compose deployment and /health smoke test
6	Agile Software Development Workflow using Jira	Scrum project with epics, stories, tasks, sprints and dependencies
7	Continuous Monitoring using Prometheus	Flask metrics, /metrics endpoint, PromQL, Grafana dashboard with CPU, memory and disk

Table 1.1: Lab experiments and their application in CivicFix

2. Requirements and Agile Planning

2.1 User Roles

Role	Main Functionality
Citizen	Register/login, report a civic issue with description, location (address/GPS) and photo, receive a unique reference number, track complaints publicly by reference number and view own complaints with history
Municipal Staff	Dashboard of pending, open, resolved and overdue complaints; verify or reject complaints; assign to a department; update status; record resolution; search, filter and export complaints as CSV
Administrator	All staff functions plus managing departments, complaint categories, users, roles and account activation

Table 2.1: User roles

2.2 Functional Requirements

- Citizens can register and log in to the system.
- Citizens can report an issue with category, title, description, address (optional GPS coordinates), priority and an optional photo (PNG/JPG/GIF/WEBP, maximum 5 MB).
- Every complaint receives a unique reference number of the form CF-YYYY-NNNNN.
- Anyone can track a complaint by its reference number and see its status timeline; a citizen can view only their own complaints in detail.
- A citizen can have at most 5 open complaints at a time.
- Municipal staff can verify or reject (with a mandatory reason) a new complaint.
- Municipal staff can assign a verified complaint to a department, start work, and mark it resolved with a mandatory resolution note. Steps cannot be skipped.
- Every status change is recorded in the complaint history with time, note and the staff member.
- Open complaints older than the 7-day SLA are automatically flagged OVERDUE.
- Staff can search and filter complaints and download the register as a CSV file.
- Administrators can add departments and categories (a category used by complaints cannot be deleted), create users, change roles and activate or deactivate accounts.

2.3 Non-Functional / DevOps Requirements

- The application should run in a consistent environment through Docker.
- Flask and PostgreSQL should communicate as interconnected services through Docker Compose, with persistent database and photo storage.
- Jenkins should automate source retrieval, building and testing, with deployment and a smoke test added to the extended pipeline.
- Jira should provide an Agile/Scrum workflow for development tasks.

- Prometheus should collect application metrics exposed by the Flask application and server metrics from node-exporter, and Grafana should visualise them.

2.4 Scrum-Style Workflow

CivicFix used Jira as the Agile planning and tracking tool. The backlog was prepared as five epics (Citizen Reporting & Tracking, Staff Verification & Resolution, Administration & Master Data, DevOps Pipeline and Monitoring) with 20 stories and tasks, estimated in story points (55 in total) and divided into three sprints (file jira/CivicFix_Jira_Import.csv in the repository). A Scrum-style workflow was followed in which each issue moves through the following stages:

To Do
↓
In Progress
↓
In Review
↓
Done

Features were developed using the Git/GitHub workflow, tested, pushed to the repository, reviewed through the repository workflow where applicable and then verified through Jenkins; the matching Jira issue is moved to Done once that verification succeeds.

2.5 Agile-to-DevOps Traceability

Stage	Tool / Practice	Outcome
Planning	Jira	Epics, stories and tasks defined, estimated and planned into sprints
Development	Git / GitHub	Source code versioned on feature branches and merged through pull requests
Verification	Jenkins	Automated build and 27 unit tests executed on every change to main
Deployment	Docker / Docker Compose	Application deployed in containers and checked through /health
Monitoring	Prometheus / Grafana	Application and server metrics collected and visualised
Completion	Jira	Task moved to Done after successful verification

Table 2.2: Agile and DevOps workflow traceability

3. System Design

3.1 Architecture

CivicFix consists of a server-rendered frontend, a Python Flask backend and a PostgreSQL database. The application and database are deployed in containers. Jenkins manages the automated delivery workflow, Jira manages Agile planning, and Prometheus with Grafana monitors the running application and the host server. The overall integrated workflow is:

Developer
↓
Git / GitHub (feature branch → pull request → main)
↓
GitHub push + Jenkins SCM polling
↓
Jenkins CI/CD: Checkout · Build Test Image · Unit Tests · Build Docker Image · Deploy · Smoke Test
↓
Docker Compose
↓
Flask (Gunicorn) + PostgreSQL
↓
/metrics + node-exporter
↓
Prometheus → Grafana

Jira → Agile planning and task tracking

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	Jinja2 templates, HTML, CSS, small JavaScript	Web interface for citizens, staff and administrators; browser geolocation for “use my location”
Backend	Python 3.12, Flask, SQLAlchemy, Gunicorn	Application logic, business rules, JSON API
Database	PostgreSQL 16 (SQLite for local tests)	Stores users, departments, categories, complaints and history
File storage	Docker named volume (uploads)	Stores complaint photographs
Testing	pytest	Automated tests for authentication, workflow, business rules, admin, CSV and metrics
Version Control	Git and GitHub	Branches, commits, pull requests, remote source code
Containerization	Docker (multi-stage)	Packages the Flask application and dependencies
Orchestration	Docker Compose	Runs web, db, prometheus, node-exporter and grafana
CI/CD	Jenkins (pipeline as code, configuration as code)	Automates tests, Docker build, deployment and smoke test

Layer	Technology	Purpose
Agile Management	Jira	Plans and tracks development tasks
Monitoring	Prometheus, node-exporter, Grafana	Application metrics from /metrics and server CPU, memory and disk

Table 3.1: Technology stack

3.3 Database Design

The relational schema has five tables. The complaint table references the citizen, category, department and the staff member it is assigned to; complaint_update stores the full status history so that the citizen can see a timeline.

Table	Key columns	Relationships
user	id, username (unique), full_name, email, password_hash, role (citizen/staff/admin), active	—
department	id, name (unique)	1 department → many categories
category	id, name (unique), department_id	Default department of a complaint type
complaint	id, ref_no (unique), title, description, address, latitude, longitude, photo, status, priority, resolution_note, created_at, resolved_at	citizen_id → user, category_id → category, department_id → department, assigned_to_id → user
complaint_update	id, complaint_id, status, note, by_user_id, created_at	many updates → 1 complaint

Table 3.2: Database tables

3.4 Complaint Lifecycle

A complaint follows a strict state machine, enforced in the model so that steps cannot be skipped:

SUBMITTED
↓
VERIFIED (or REJECTED with a reason)
↓
ASSIGNED (department required)
↓
IN PROGRESS
↓
RESOLVED (resolution note required)

An open complaint (not resolved or rejected) whose age reaches the 7-day SLA is shown as OVERDUE on the staff dashboard and counted in the overdue metric.

3.5 Service Design

Service	Responsibility
web (Flask + Gunicorn)	Runs CivicFix on port 5000 and exposes pages, /api, /health and /metrics; photos stored in the uploads volume
db (PostgreSQL 16)	Stores data in the named volume pgdata; a pg_isready health check gates the web service
prometheus	Scrapes web:5000, node-exporter:9100 and itself every 10 seconds and supports PromQL analysis
node-exporter	Exposes host CPU, memory and disk metrics
grafana	Shows the provisioned dashboard “CivicFix – Application & Server Monitoring”

Table 3.3: Service design

3.6 Monitoring Design

The prometheus-flask-exporter package was integrated with Flask to expose HTTP metrics (flask_http_request_total and a request-duration histogram) through the /metrics endpoint. Custom business metrics were added in app/metrics.py and are computed from the database at every scrape: civicfix_complaints{status}, civicfix_complaints_overdue, civicfix_open_complaints_by_department{department}, civicfix_complaints_submitted_total and civicfix_complaints_resolved_total. Prometheus and node-exporter were added to the Docker Compose environment and reach the Flask service through the Compose network. The Grafana dashboard combines application status, open and overdue complaints, complaints by status and department, request rate, p95 latency and server CPU, memory and disk usage.

4. Application Implementation

4.1 Application Features

CivicFix provides separate functionality for citizens, municipal staff and administrators. Business rules: complaints receive a unique reference CF-YYYY-NNNNN; at most 5 open complaints per citizen; photos must be images up to 5 MB; rejection and resolution require a written note; workflow steps cannot be skipped; and open complaints older than 7 days are shown as OVERDUE. Passwords are stored as salted hashes, and every route is protected by role-based access control (a citizen cannot open staff or admin pages, and cannot view another citizen's complaint).

4.2 Citizen Workflow

Register / Login
↓
Report Issue (category, description, location, photo)
↓
Receive Reference Number (CF-YYYY-NNNNN)
↓
Track Status / View Timeline
↓
See Resolution

4.3 Municipal Staff Workflow

View Dashboard (pending / overdue)
↓
Verify or Reject Complaint
↓
Assign to Department
↓
Start Work → Update Status
↓
Record Resolution
↓
Search / Filter / Export CSV

4.4 Administrator Workflow

Manage Departments
↓
Manage Categories (mapped to departments)
↓
Create Users / Change Roles / Activate-Deactivate

4.5 Project Repository and Application Structure

The Git repository maintains the Flask backend, HTML templates, static files, Docker configuration, Jenkinsfile, tests, Jira backlog and monitoring configuration. These project files

were committed and pushed to the GitHub repository [https://github.com/<your-username>/civicfix] as part of Experiment 1.

```
civicfix/
├── app/                    Flask application
│   ├── __init__.py         app factory
│   ├── models.py           User, Department, Category, Complaint,
ComplaintUpdate + workflow rules
│   ├── routes.py           auth, citizen, staff, admin pages, JSON API, CSV
export
│   ├── metrics.py          Prometheus business metrics
│   ├── seed.py             master data, default admin, demo data
│   ├── templates/          base, index, track, citizen/, staff/, admin/
│   └── static/style.css
├── tests/                  pytest suite (test_app.py, conftest.py)
├── monitoring/             prometheus/ and grafana/ (dashboard + provisioning)
├── jenkins/                Jenkins image, plugins.txt, casc.yaml (configuration
as code)
├── jira/                   CivicFix_Jira_Import.csv (5 epics, 20 stories/tasks)
├── docs/                   LAB_GUIDE.md, screenshots/
├── Dockerfile              multi-stage image (test + production)
├── docker-compose.yml      web, db, prometheus, node-exporter, grafana
├── Jenkinsfile             CI/CD pipeline
├── requirements.txt / requirements-dev.txt
└── wsgi.py
```

Figure 4.1: CivicFix project repository structure

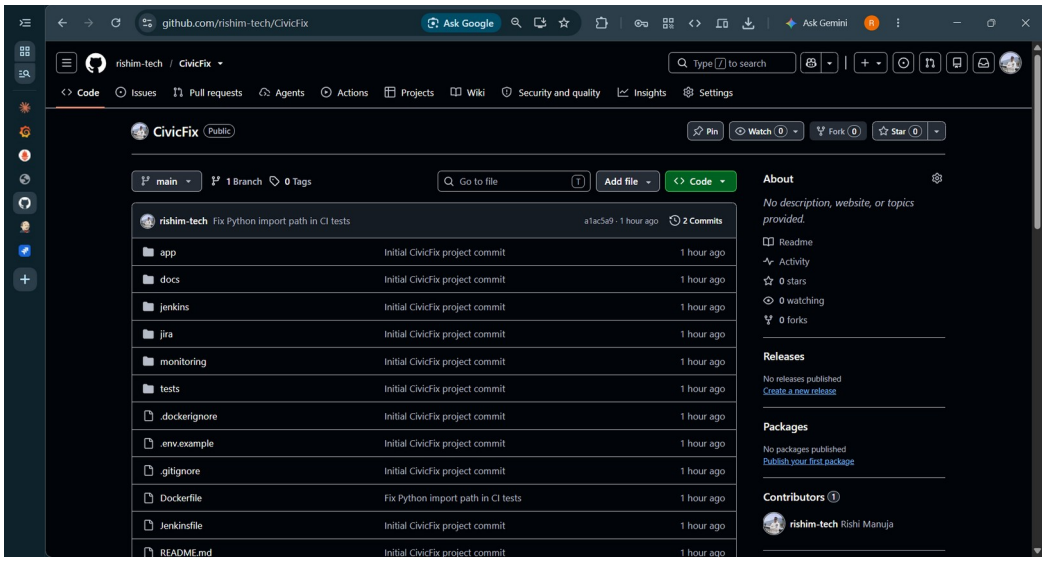


Figure 4.2: CivicFix GitHub repository on the main branch

5. DevOps Implementation

5.1 Version Control Using Git and GitHub

Git was used as the distributed version-control system for CivicFix. The project repository maintains the Flask backend, HTML templates, static files, Docker configuration, Jenkinsfile, tests, Jira backlog and monitoring configuration. Changes were staged and committed using Git, while GitHub was used as the remote repository. Each part of the project was developed on its own branch (feature/app-core, feature/tests, feature/docker, feature/monitoring, feature/jenkins, docs/agile-and-guide) and merged into the main line with --no-ff so that every merge is visible in the history. git log --oneline --graph was used to inspect commit history.

A new feature was developed on a separate branch and merged into main through Pull Request #1. To demonstrate rollback, a commit that raised the open-complaint limit per citizen from 5 to 10 was pushed to main; the Jenkins build failed on the unit test that enforces the agreed limit, and the change was rolled back with git revert, after which the next build passed and redeployed the application.

LO2: "Use Git effectively for version control, source code management and collaborative team management."

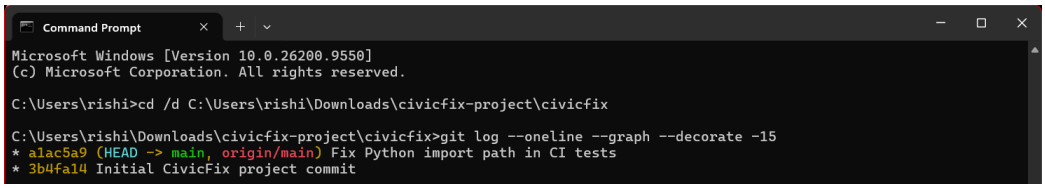


Figure 5.1: Git log --oneline --graph showing feature branches merged into main

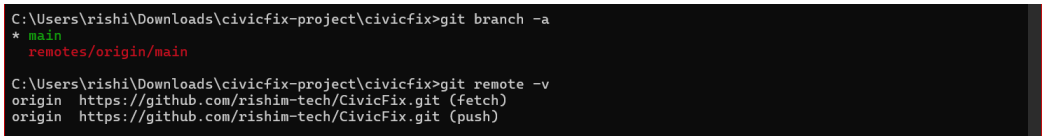


Figure 5.2: Git branches and remote repository

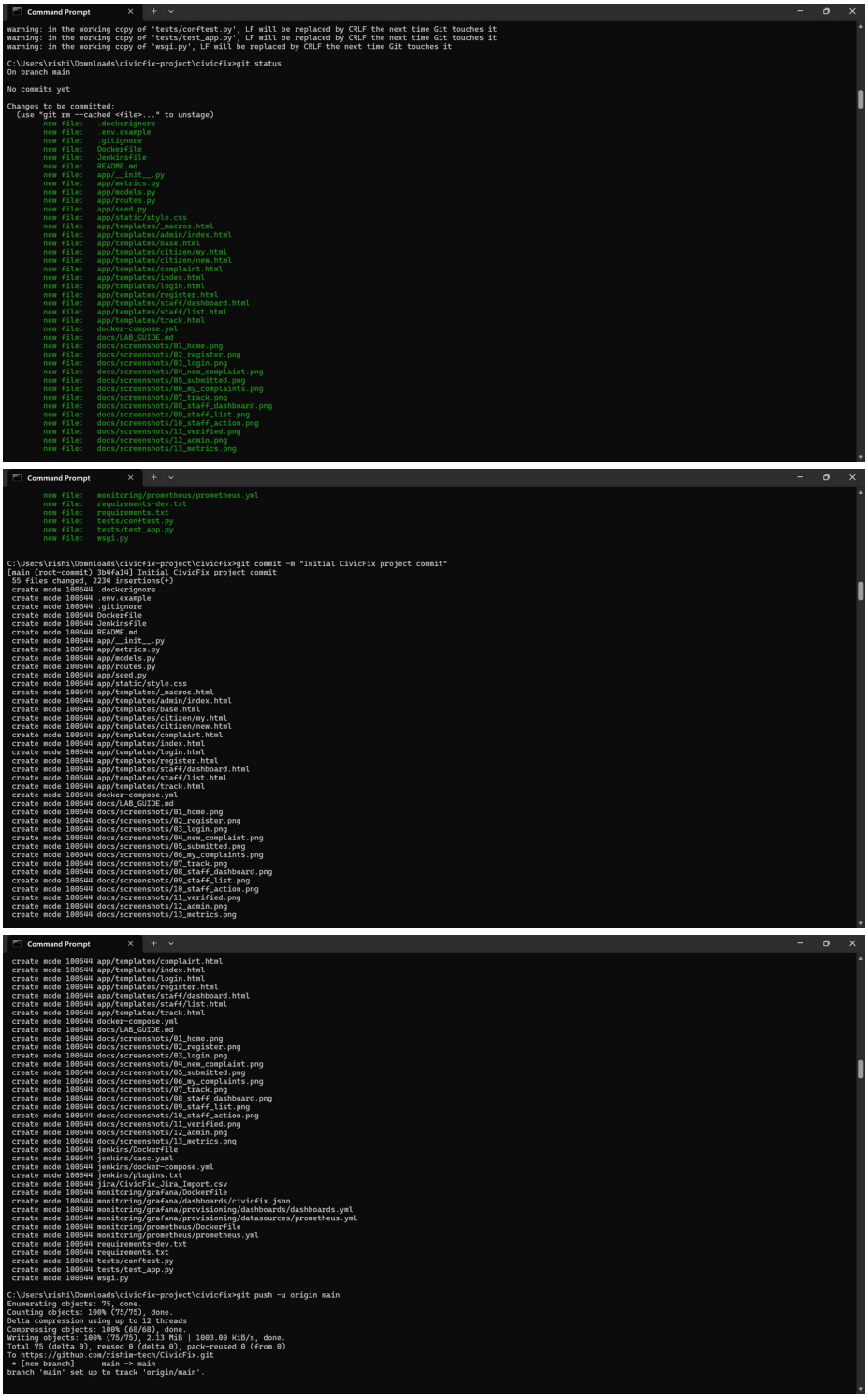


Figure 5.3: Git status, commit and push of the CivicFix project files to GitHub

Evidence note: A separate merged pull request screenshot is not included in this submission. Version-control evidence is represented by the repository and commit-history screenshots.

Figure 5.4: Version-control workflow note (no separate pull-request screenshot available)

Figure 5.5: GitHub commit history on main (screenshot not included in this submission)

5.2 Containerization Using Docker

Docker packages an application together with its dependencies, libraries, runtime and configuration into a consistent environment. A Docker image acts as the template, while a container is a running instance created from that image. For CivicFix, a multi-stage Dockerfile was created using the python:3.12-slim base image. The base stage sets the working directory, installs the required packages and copies the source code. The test stage adds pytest and the tests and is used by Jenkins to run the unit tests. The production stage runs as a non-root user, exposes port 5000, defines a HEALTHCHECK on /health and starts the application with Gunicorn.

```
FROM python:3.12-slim AS base
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
COPY wsgi.py .

FROM base AS test
COPY requirements-dev.txt .
RUN pip install --no-cache-dir -r requirements-dev.txt
COPY tests ./tests
CMD ["pytest", "-v"]

FROM base AS production
RUN useradd --create-home civicfix && mkdir -p /data/uploads && chown -R civicfix
/app /data
USER civicfix
EXPOSE 5000
HEALTHCHECK --interval=10s CMD python -c "import urllib.request as u;
u.urlopen('http://localhost:5000/health')"
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "1", "--threads", "4",
"wsgi:app"]
```

Figure 5.6: Multi-stage Dockerfile (abridged)

The image was built with docker build -t civicfix . and a container was created from it, mapping port 5000 of the container to port 5001 of the host (docker run -d -p 5001:5000 --name civicfix-single civicfix). docker images, docker ps, docker logs, docker stop, docker start, docker inspect and docker rm were used for container management and troubleshooting. Running without a PostgreSQL service, the container falls back to SQLite, so the single-container demonstration works on its own.

LO4: “Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.”

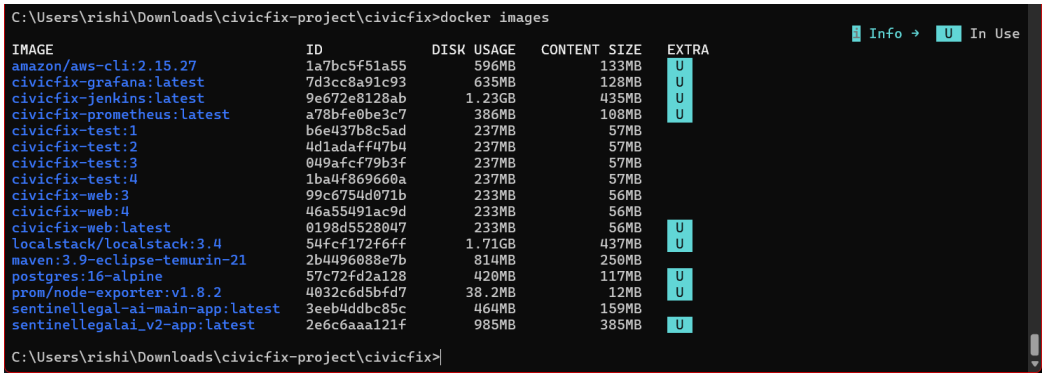


Figure 5.7: Docker image build and images list

Figure 5.8: Docker container lifecycle and logs (screenshot not included in this submission)

Figure 5.9: CivicFix container health check (screenshot not included in this submission)

5.3 Multi-Service Deployment Using Docker Compose

Docker Compose extends Docker to multi-service applications by allowing related services to be declared in a single YAML configuration file. CivicFix requires an application server and a database, and the monitoring stack runs alongside them, so docker-compose.yml was created with five services: web runs the Flask application, db uses the postgres:16-alpine image, and prometheus, node-exporter and grafana provide monitoring. The Flask container communicates with PostgreSQL using the Compose service name db and the DATABASE_URL environment variable. Named volumes provide persistent storage for the database (pgdata), the uploaded complaint photos (uploads), Prometheus and Grafana data, while a PostgreSQL health check (pg_isready) ensures that the Flask service waits for the database to become healthy before starting. docker compose config, docker compose up -d --build, docker compose ps and docker compose logs were used for validation and troubleshooting.

```
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: civicfix
      POSTGRES_USER: civicfix
      POSTGRES_PASSWORD: ${DB_PASSWORD:-civicfix123}
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U civicfix -d civicfix"]

  web:
    build: { context: ., target: production }
    ports: ["5000:5000"]
    environment:
      DATABASE_URL:
postgresql://civicfix:${DB_PASSWORD:-civicfix123}@db:5432/civicfix
    volumes: [uploads:/data/uploads]
    depends_on:
      db: { condition: service_healthy }
```

Figure 5.10: Docker Compose configuration (db and web services, abridged)

LO4: "Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools."

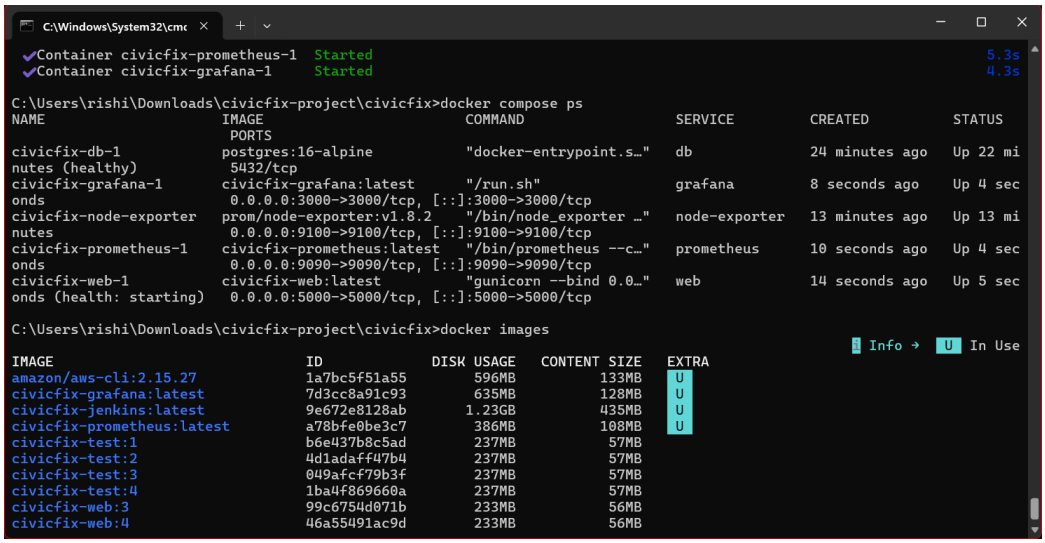


Figure 5.11: Docker Compose services, volumes, network and logs

Figure 5.12: Docker Desktop showing the civicfix Compose stack (screenshot not included in this submission)

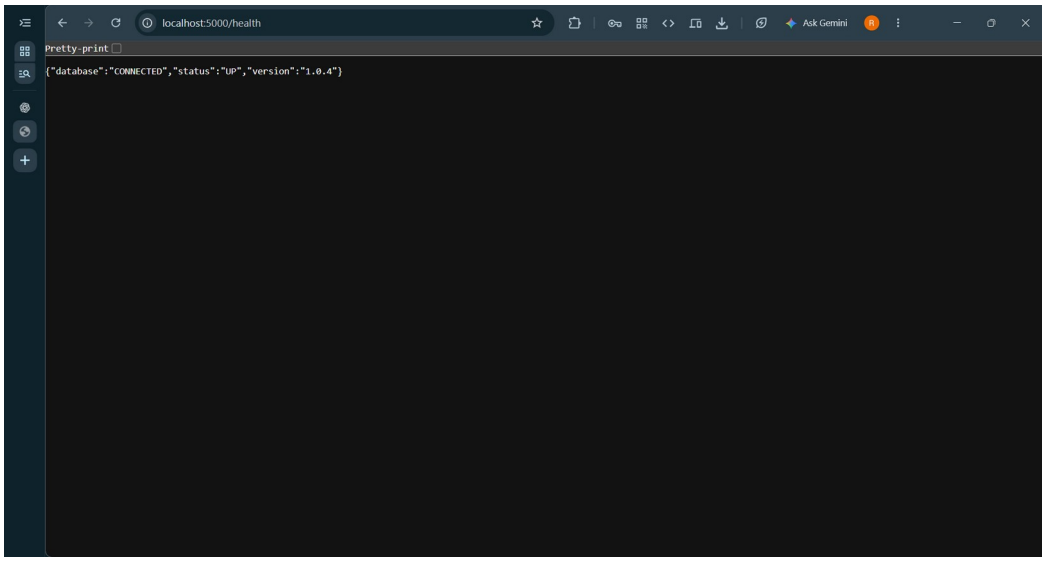


Figure 5.13: Running multi-service environment: /health reports the database CONNECTED

5.4 Continuous Integration Using Jenkins

Continuous Integration is a development practice in which developers frequently integrate changes into a shared repository and automated processes build and test those changes. Jenkins is an automation server used for CI/CD activities such as building applications, executing tests and performing deployment operations. Jenkins was run in a Docker container (jenkins/Dockerfile with the Docker CLI and Compose plugin, using the host Docker socket), and a pipeline job named CivicFix-CI-CD was connected to the GitHub repository, branch */main. The job is created automatically through Jenkins Configuration as Code (jenkins/casc.yaml), and the Jenkinsfile is stored with the project source code. The CI part of

the pipeline checks out the source code, builds the test image and runs the automated tests using pytest inside it, publishing a JUnit test report.

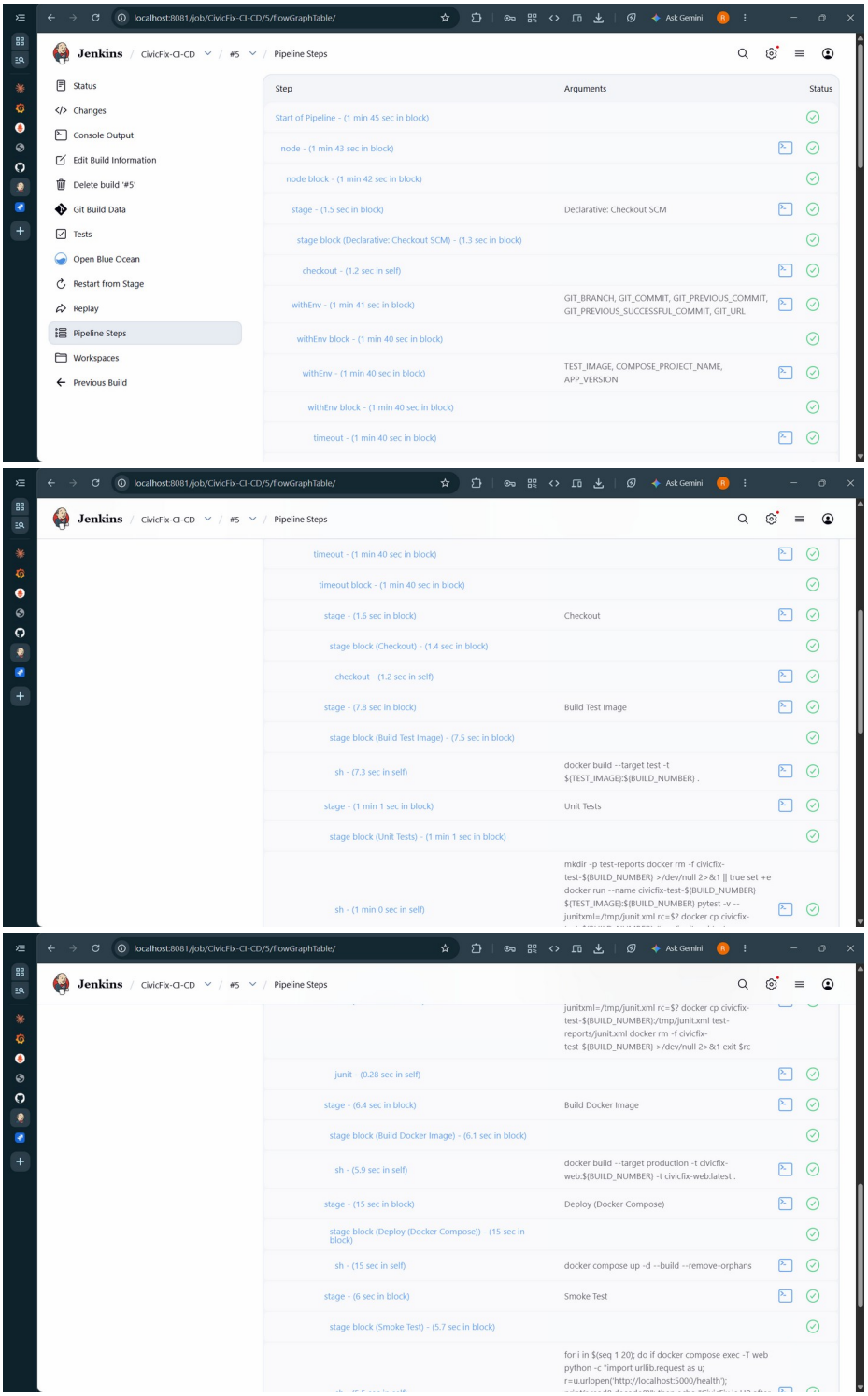
GitHub and Jenkins were connected through SCM polling: Jenkins checks the repository every 2 minutes, so new pushes and merged pull requests on main automatically start the pipeline. The job also accepts GitHub webhooks, which can be exposed through an ngrok tunnel when Jenkins runs on a laptop.

Developer
↓
Git Push / Pull Request merge
↓
GitHub
↓
SCM polling (or webhook)
↓
Jenkins
↓
Checkout + Build Test Image + Unit Tests
↓
SUCCESS / FAILURE

LO3: "To Build and Manage automated CI/CD pipeline, for application development, packaging, testing and deployment."

Figure 5.14: Jenkins dashboard with the CivicFix-CI-CD job (screenshot not included in this submission)

Figure 5.15: Jenkins pipeline configuration: GitHub repository, */main, SCM polling, Jenkinsfile (screenshot not included in this submission)


localhost:8081/job/CivicFix-CI-CD/5/flowGraphTable/
Jenkins / CivicFix-CI-CD / #5 / Pipeline Steps
Status
Changes
Console Output
Edit Build Information
Delete build '#5'
Git Build Data
Tests
Open Blue Ocean
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Step
Arguments
Status
Start of Pipeline - (1 min 45 sec in block)
node - (1 min 43 sec in block)
node block - (1 min 42 sec in block)
stage - (1.5 sec in block)
Declarative: Checkout SCM
stage block (Declarative: Checkout SCM) - (1.3 sec in block)
checkout - (1.2 sec in self)
withEnv - (1 min 41 sec in block)
GIT_BRANCH, GIT_COMMIT, GIT_PREVIOUS_COMMIT, GIT_PREVIOUS_SUCCESSFUL_COMMIT, GIT_URL
withEnv block - (1 min 40 sec in block)
withEnv - (1 min 40 sec in block)
TEST_IMAGE, COMPOSE_PROJECT_NAME, APP_VERSION
withEnv block - (1 min 40 sec in block)
timeout - (1 min 40 sec in block)

localhost:8081/job/CivicFix-CI-CD/5/flowGraphTable/
Jenkins / CivicFix-CI-CD / #5 / Pipeline Steps
timeout - (1 min 40 sec in block)
timeout block - (1 min 40 sec in block)
stage - (1.6 sec in block)
Checkout
stage block (Checkout) - (1.4 sec in block)
checkout - (1.2 sec in self)
stage - (7.8 sec in block)
Build Test Image
stage block (Build Test Image) - (7.5 sec in block)
sh - (7.3 sec in self)
docker build --target test -t ${TEST_IMAGE}:${BUILD_NUMBER} .
stage - (1 min 1 sec in block)
Unit Tests
stage block (Unit Tests) - (1 min 1 sec in block)
sh - (1 min 0 sec in self)
mkdir -p test-reports docker rm -f civicfix-test-${BUILD_NUMBER} >/dev/null 2>&1 || true set +e docker run --name civicfix-test-${BUILD_NUMBER} ${TEST_IMAGE}:${BUILD_NUMBER} pytest -v --junitxml=/tmp/junit.xml rc=$? docker cp civicfix-

localhost:8081/job/CivicFix-CI-CD/5/flowGraphTable/
Jenkins / CivicFix-CI-CD / #5 / Pipeline Steps
junitxml=/tmp/junit.xml rc=$? docker cp civicfix-test-${BUILD_NUMBER}:/tmp/junit.xml test-reports/junit.xml docker rm -f civicfix-test-${BUILD_NUMBER} >/dev/null 2>&1 exit $rc
junit - (0.28 sec in self)
stage - (6.4 sec in block)
Build Docker Image
stage block (Build Docker Image) - (6.1 sec in block)
sh - (5.9 sec in self)
docker build --target production -t civicfix-web:${BUILD_NUMBER} -t civicfix-web:latest .
stage - (15 sec in block)
Deploy (Docker Compose)
stage block (Deploy (Docker Compose)) - (15 sec in block)
sh - (15 sec in self)
docker compose up -d --build --remove-orphans
stage - (6 sec in block)
Smoke Test
stage block (Smoke Test) - (5.7 sec in block)
for i in $(seq 1 20); do if docker compose exec -T web python -c "import urllib.request as u; r=u.urlopen('http://localhost:5000/health');

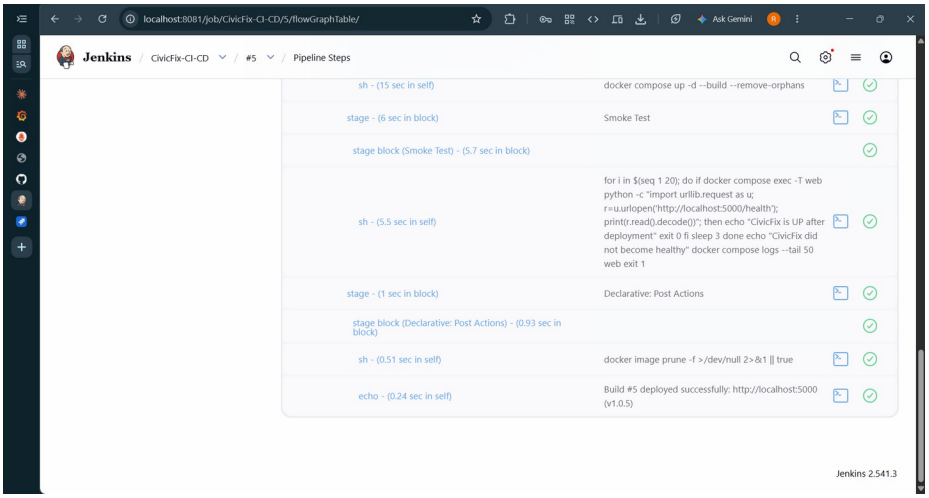


Figure 5.16: Jenkins build with all six stages successful (stage view)

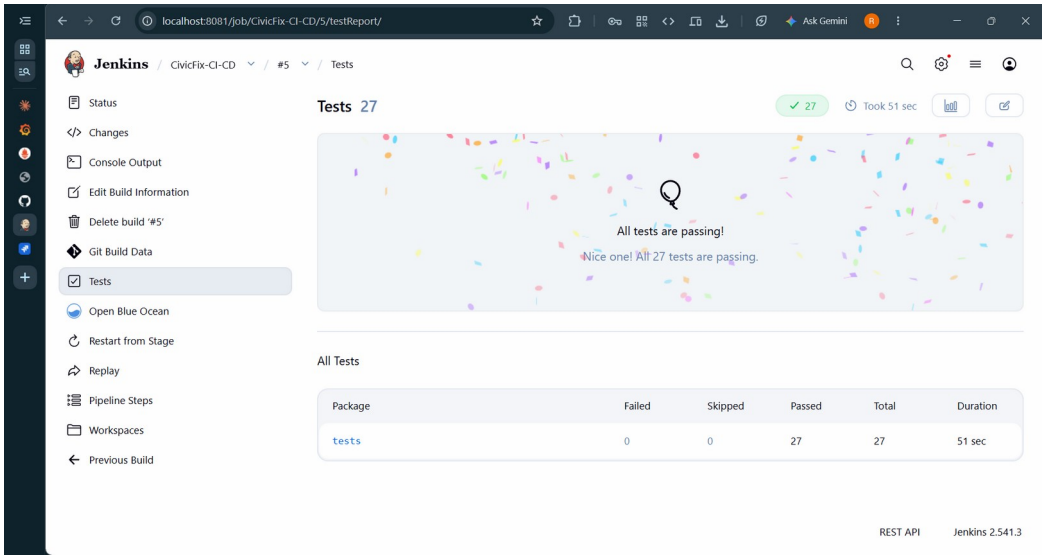


Figure 5.17: JUnit test report with 27 tests passed

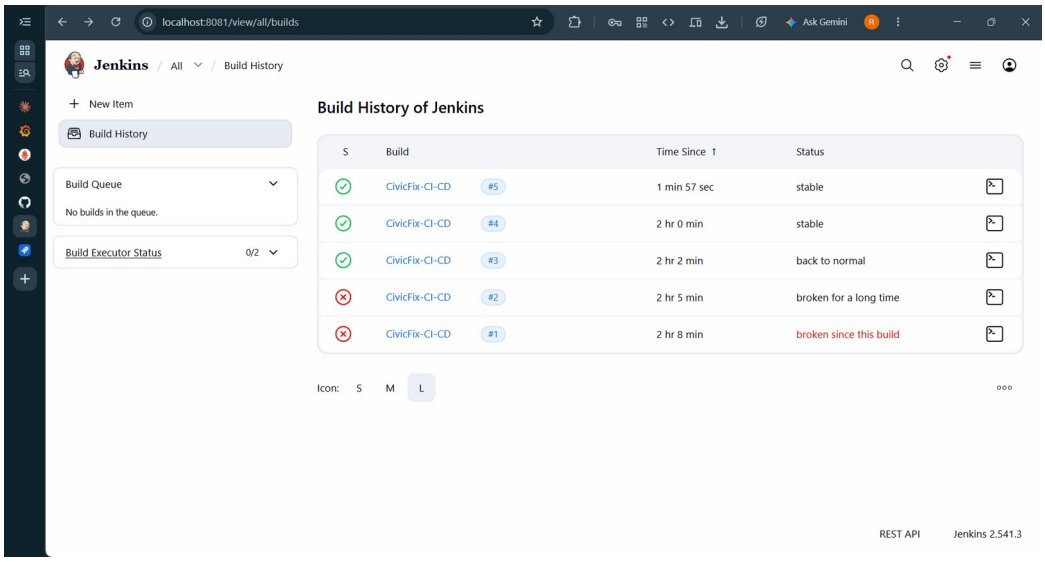


Figure 5.18: Jenkins build history

5.5 Automated Deployment Using GitHub and Jenkins

This extends Continuous Integration toward Continuous Deployment. The Jenkins pipeline created in Experiment 4 was extended to perform Docker image building, deployment and a smoke test. The resulting workflow became:

Git Push
↓
GitHub
↓
SCM polling / Webhook
↓
Jenkins
↓
Checkout
↓
Build Test Image
↓
pytest (Unit Tests)
↓
Build Docker Image
↓
Docker Compose Deployment
↓
Smoke Test (/health)
↓
Updated CivicFix Application

The pipeline first retrieves the latest source code and builds the test image. It then runs the CivicFix automated tests. Only after the testing stage succeeds does Jenkins proceed to build the production Docker image (tagged with the build number and latest), deploy the application using Docker Compose with APP_VERSION=1.0.<build number>, and check that /health responds from inside the web container. This makes deployment part of the automated pipeline instead of requiring the developer to repeat the complete sequence manually.

Four builds were run against main to demonstrate the pipeline. [Fill the “Result” column from your own Jenkins history.]

Build	Started by	Change	Result
#1	User (Build Now)	main before the pull request	[e.g. SUCCESS: 27 tests passed, deployed v1.0.1]
#2	SCM change	Merge of Pull Request #1	[e.g. SUCCESS: tests passed, deployed v1.0.2]
#3	SCM change	“Raise open-complaint limit from 5 to 10”	[e.g. FAILURE: 1 failed; Docker build and deploy skipped]
#4	SCM change	Revert of the breaking change	[e.g. SUCCESS: tests passed, deployed v1.0.4]

Table 5.1: Jenkins builds of CivicFix-CI-CD

The failing build shows that a faulty change is stopped at the test stage and never reaches the running application, and the following build shows the rollback being deployed automatically.

LO3: "Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment."

Figure 5.19: Blue Ocean activity for builds #1–#4 (screenshot not included in this submission)

Figure 5.20: Console output: "Started by an SCM change" after the pull request merge (screenshot not included in this submission)

Figure 5.21: Build #3: Unit Tests failed, image build and deployment skipped (screenshot not included in this submission)

Figure 5.22: Build #3 JUnit report showing the failing test (screenshot not included in this submission)

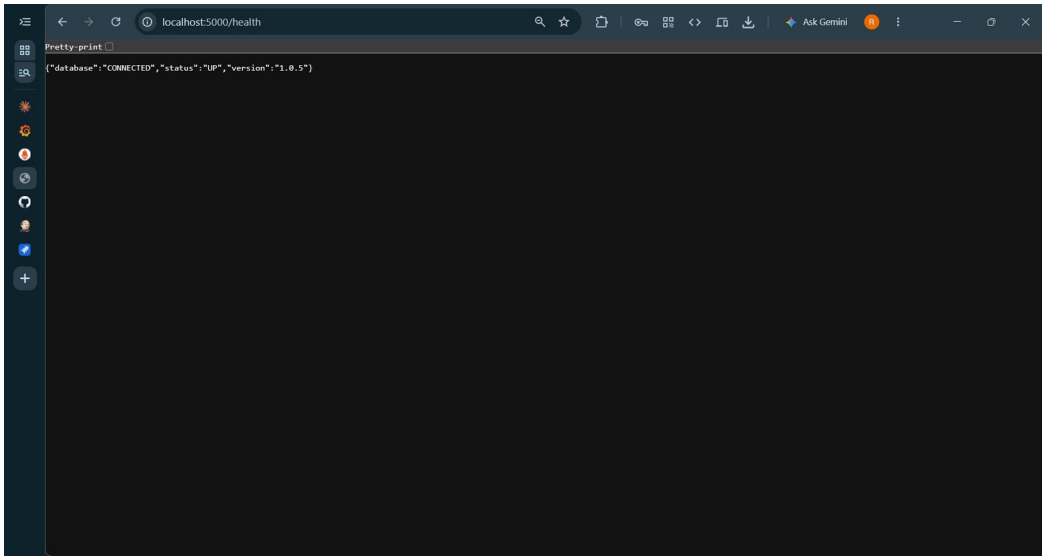


Figure 5.23: Deployed version v1.0.5 reported by /health after the final build

5.6 Agile Software Development Workflow Using Jira

Jira was used as the Agile planning and tracking tool. The experiment connects Agile development, Jira, Git/GitHub, Jenkins and Docker-related development activities into one practical workflow. A Scrum project named CivicFix (key CF) was planned with the following epics, imported from jira/CivicFix_Jira_Import.csv:

Epic	Stories / Tasks	Sprint
Citizen Reporting & Tracking	Register/login; submit complaint with description, location and photo; unique reference number; track complaint and history	Sprints 1–2
Staff Verification & Resolution	Verify/reject; assign to department; update status; record resolution; overdue flag and dashboard	Sprint 2
Administration & Master Data	Manage departments and categories; manage users and roles; export CSV	Sprints 1–3
DevOps Pipeline	Git repository and branches; Dockerfile; docker-compose.yml; pytest unit tests; Jenkins pipeline	Sprints 1–3

Epic	Stories / Tasks	Sprint
Monitoring	Expose /metrics; Prometheus and node-exporter; Grafana dashboard	Sprint 3

Table 5.2: CivicFix product backlog (epics)

Each story carries a priority and a story-point estimate. Dependencies were recorded with "is blocked by" links: "Assign a verified complaint to a department" is blocked by "Verify or reject complaints", and "Create Jenkins pipeline" is blocked by "Write pytest unit tests". A Scrum-style workflow was followed (To Do → In Progress → In Review → Done). A feature task was implemented on a feature branch, tested and pushed to the repository, reviewed through the repository workflow where applicable and verified by Jenkins, which is the point at which its Jira issue moves to Done.

Jira Task
↓
Development
↓
Git / GitHub
↓
Jenkins Verification
↓
Docker Deployment
↓
Jira Done

LO1: "Apply Agile methodologies and use Jira for sprint planning, task tracking, and integrating with DevOps workflows."

Epic	Issues	Story points	Sprints
Citizen Reporting & Tracking	4	13	1, 2
Staff Verification & Resolution	5	14	2
Administration & Master Data	3	7	1, 2, 3
DevOps Pipeline	5	14	1, 2, 3
Monitoring	3	7	3
Total	20	55	3 sprints (17 + 24 + 14 points)

Table 5.3: Epic breakdown with story points (from jira/CivicFix_Jira_Import.csv)

Issue	Link	Depends on
CF-11 Assign a verified complaint to a department	is blocked by	CF-10 Verify or reject complaints
CF-22 Create Jenkins pipeline (Jenkinsfile)	is blocked by	CF-21 Write pytest unit tests

Table 5.4: Issue dependencies

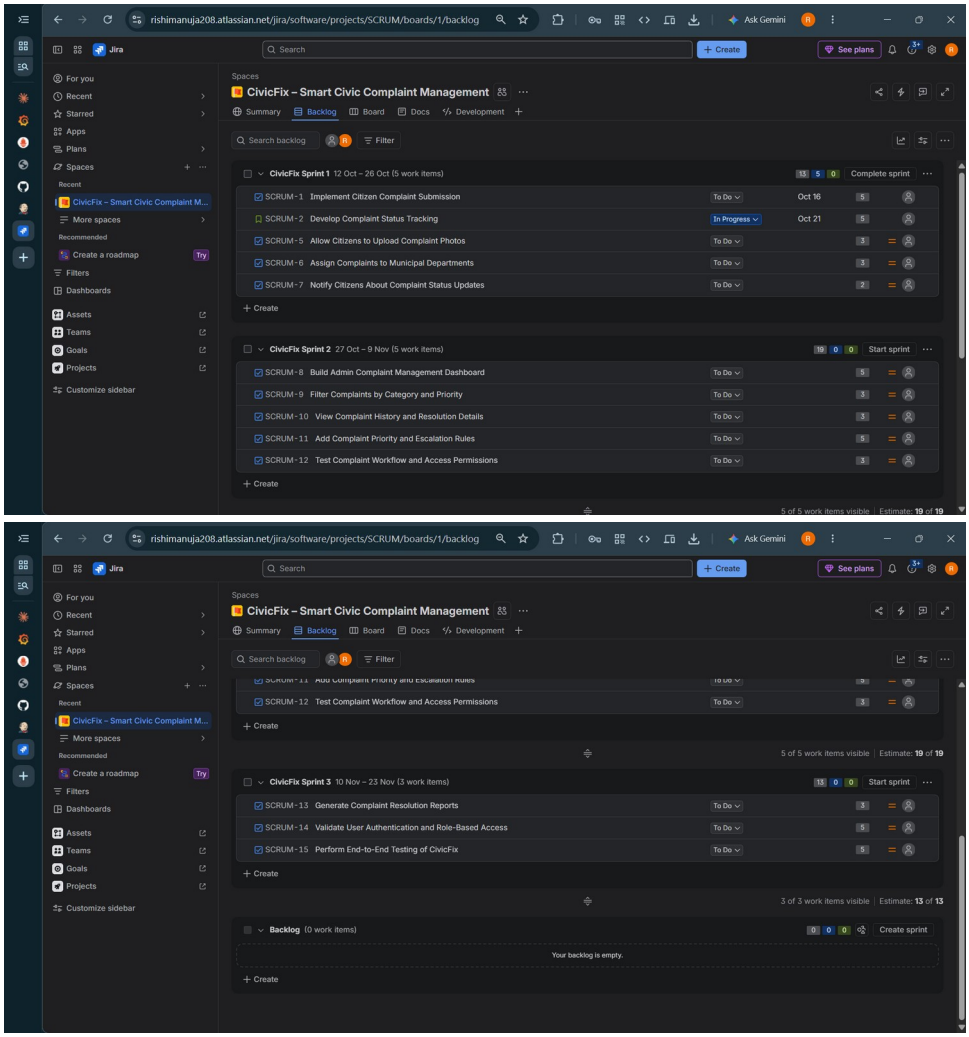


Figure 5.24: Jira backlog with Sprint 1, Sprint 2 and Sprint 3

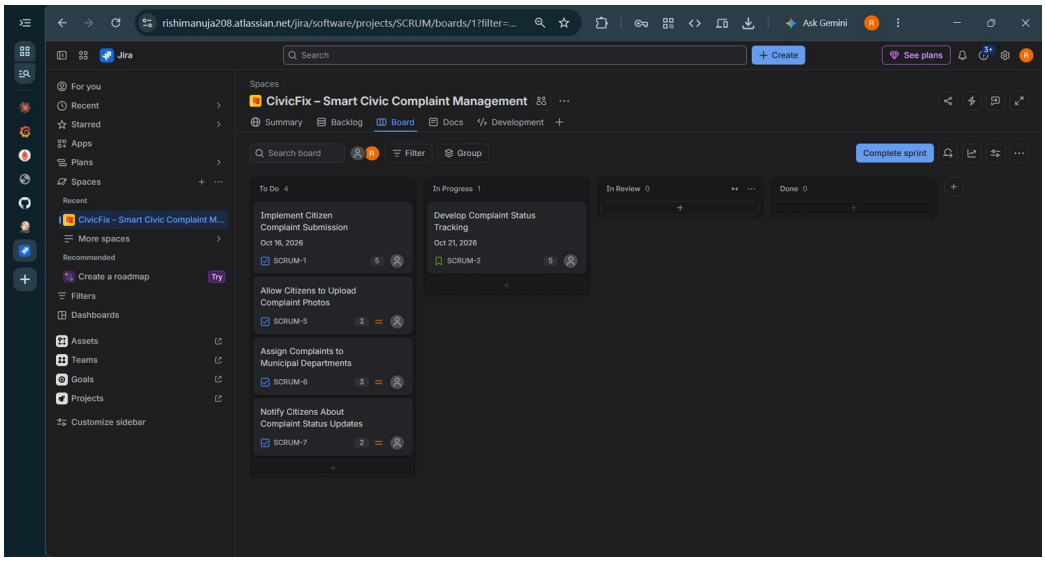


Figure 5.25: Sprint 1 board with To Do, In Progress, In Review and Done

Figure 5.26: Timeline of the five epics with dependencies (screenshot not included in this submission)

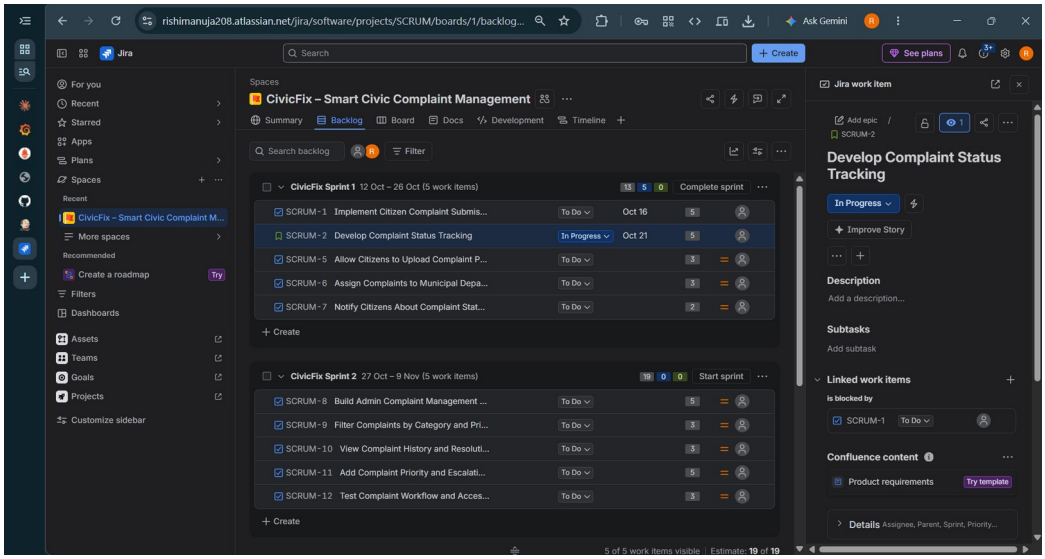


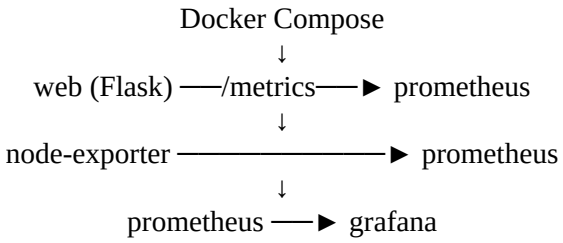
Figure 5.27: CF-11 “is blocked by” CF-10

Figure 5.28: Sprint burndown chart (screenshot not included in this submission)

5.7 Continuous Monitoring Using Prometheus

Continuous monitoring involves observing applications, servers, networks and infrastructure so that failures and performance-related problems can be detected after deployment. For CivicFix, Prometheus and Grafana were selected as the monitoring technologies. Prometheus collects and stores time-series metrics, while Grafana visualises them through dashboards. The prometheus-flask-exporter package was integrated with the Flask application and, together with custom CivicFix metrics, exposed application monitoring information through the /metrics endpoint. node-exporter was added to monitor the server’s CPU, memory and disk. A prometheus.yml configuration was created with three scrape jobs (civicfix, node and prometheus, every 10 seconds), and Prometheus was added as another service in the Docker Compose configuration. Prometheus accesses the Flask service through web:5000 on the Compose network and periodically scrapes the /metrics endpoint.

The Prometheus target page was used to verify that the CivicFix application was UP. PromQL queries were used to analyse the application: civicfix_complaints shows complaints by status, civicfix_complaints_overdue shows SLA breaches and rate(flask_http_request_total[1m]) shows HTTP requests by response status. Grafana opens on the provisioned dashboard “CivicFix – Application & Server Monitoring” with application status, open, overdue and awaiting-verification complaints, complaints by status and department, request rate, p95 latency, CPU, memory and disk.



LO5: “Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.”

```yaml
global:
  scrape_interval: 10s
  evaluation_interval: 10s
scrape_configs:
  - job_name: civicfix
    metrics_path: /metrics
    static_configs:
      - targets: ["web:5000"]
  - job_name: node
    static_configs:
      - targets: ["node-exporter:9100"]
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]
```

Figure 5.29: Prometheus configuration

```
# HELP flask_exporter_info Information about the Prometheus Flask exporter
# TYPE flask_exporter_info gauge
flask_exporter_info{version="0.23.2"} 1.0
# HELP flask_http_request_duration_seconds Flask HTTP request duration in seconds
# TYPE flask_http_request_duration_seconds histogram
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.005",method="GET",status="200"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.01",method="GET",status="200"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.025",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.05",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.075",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.1",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.25",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.5",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="0.75",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="1.0",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="2.5",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="5.0",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="7.5",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="10.0",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_bucket{endpoint="main.index",le="+Inf",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_count{endpoint="main.index",method="GET",status="200"} 5.0
flask_http_request_duration_seconds_sum{endpoint="main.index",method="GET",status="200"} 0.02572965399997429
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.005",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.01",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.025",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.05",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.075",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.1",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.25",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.5",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="0.75",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="1.0",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="2.5",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="5.0",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="7.5",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="10.0",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_bucket{endpoint="static",le="+Inf",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_count{endpoint="static",method="GET",status="200"} 2.0
flask_http_request_duration_seconds_sum{endpoint="static",method="GET",status="200"} 0.002033357000016167
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.005",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.01",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.025",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.05",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.075",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.1",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.25",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.5",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="0.75",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="1.0",method="GET",status="200"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="main.register",le="2.5",method="GET",status="200"} 1.0
```

Figure 5.30: Raw /metrics endpoint of CivicFix (captured from the running application)

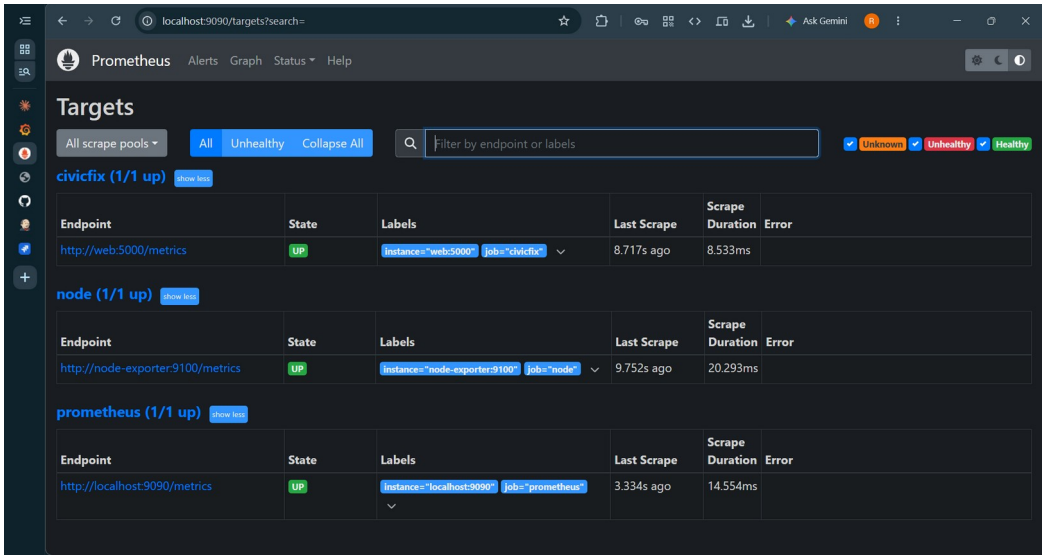


Figure 5.31: Prometheus target status showing civicfix, node and prometheus UP

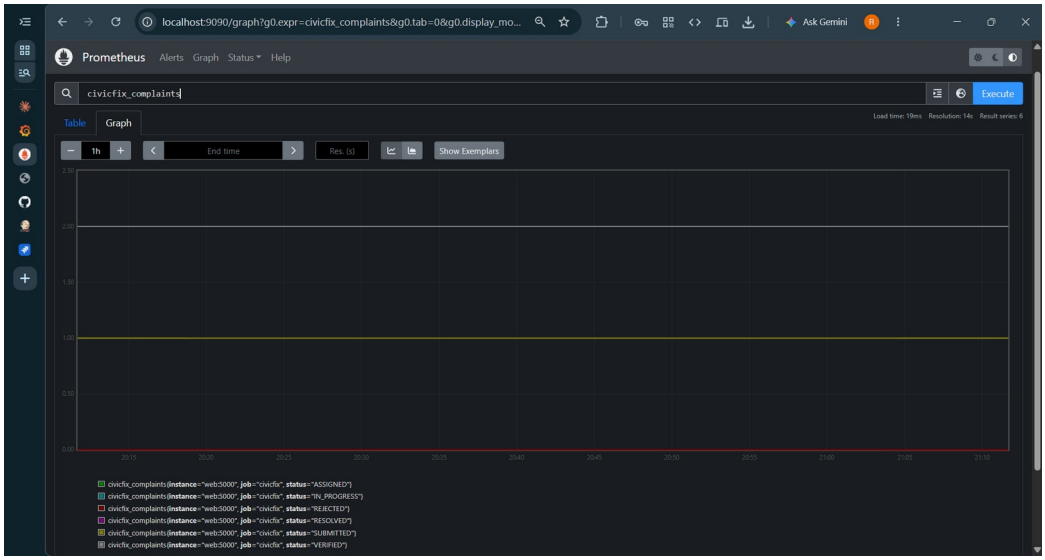


Figure 5.32: Prometheus query: civicfix_complaints by status

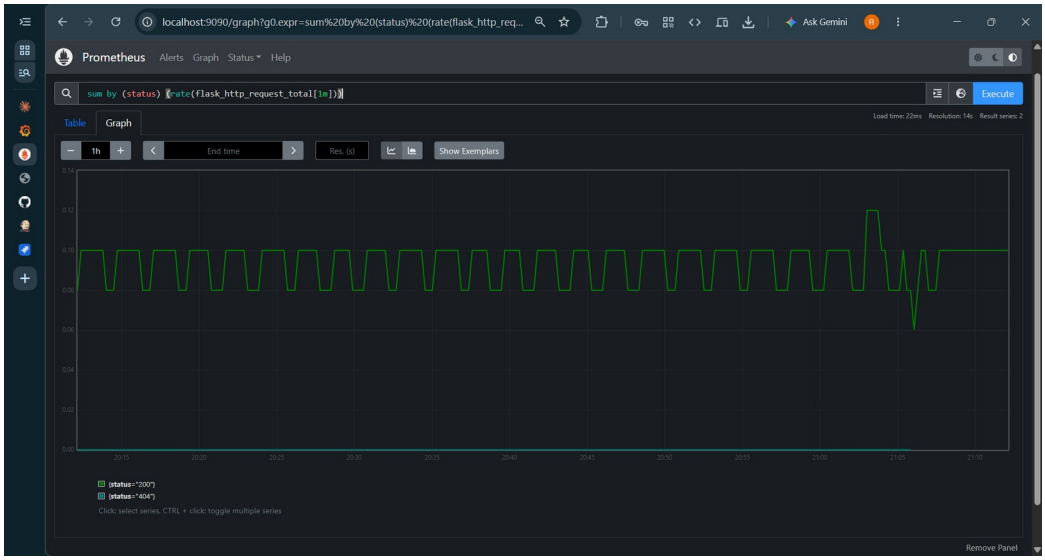


Figure 5.33: Prometheus HTTP request metrics: request rate by status



Figure 5.34: Grafana dashboard: application and server (CPU, memory, disk) monitoring

6. Testing

6.1 Automated Testing

Testing was incorporated into the Jenkins Continuous Integration pipeline. A pytest suite (tests/test_app.py) of 27 tests covers the health endpoint and API, seeded master data, registration and login, duplicate usernames and wrong passwords, role checks, complaint submission with reference-number generation, photo upload and file-type validation, the open-complaint limit, public tracking, access control between citizens, the full staff workflow (verify → assign → start → resolve), prevention of skipped steps, mandatory notes, overdue detection, CSV export, filtering, administrator functions (departments, categories, roles, deactivation) and the metrics endpoint. The Jenkinsfile runs the tests inside the test stage of the Dockerfile and publishes a JUnit report, so the tests are executed as part of the CI workflow rather than only as a manual local activity.

Area	Tests	Examples
Health / API / seed	3	database CONNECTED; four departments seeded; /api/stats counts
Authentication	4	register + login, duplicate username, wrong password, redirect to login
Citizen workflow	7	reference number, photo saved, non-image rejected, short title, 5-complaint limit, public tracking, cross-citizen access denied
Staff workflow	8	full resolution, skipped steps blocked, department required, notes required, overdue flag, CSV export, filters, citizen blocked
Administration	4	add department/category, change role, staff blocked, deactivated user cannot log in
Monitoring	1	/metrics exposes civicfix_* and flask_http_* metrics

Table 6.1: Test coverage summary (27 tests)

6.2 CI Verification

The CI workflow was verified through the sequence Developer → Git Push → GitHub → SCM polling → Jenkins → Build Test Image + Unit Tests → SUCCESS / FAILURE. The deliberately broken build confirmed that a failing test stops the pipeline: the Docker build, deployment and smoke test stages were skipped. The deployment extension in Experiment 5 additionally required the testing stage to succeed before Docker image building, Docker Compose deployment and the /health smoke test proceeded.

6.3 Monitoring Verification

Monitoring verification was performed by checking the Prometheus target page and confirming that the CivicFix application, node-exporter and Prometheus were UP. PromQL was then used to inspect complaint and HTTP request metrics, and the Grafana dashboard was checked for application and server panels. This verified that monitoring was integrated with the deployed application.

6.4 Testing Evidence

This report contains screenshots and command output of Git, Docker, Docker Compose, Jenkins and Prometheus/Grafana activities. These provide practical evidence of the implementation and verification stages described in the experiments.

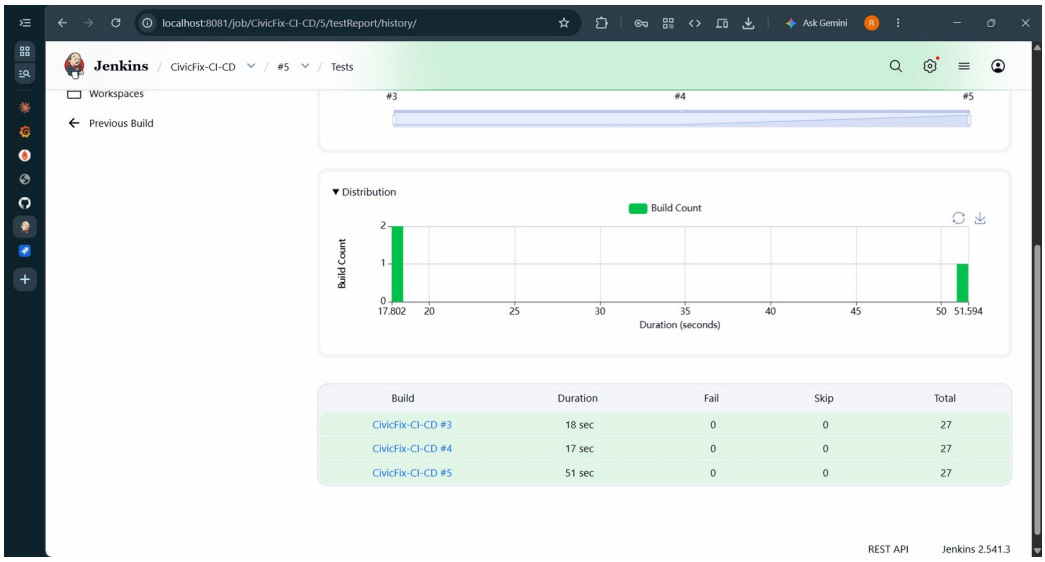
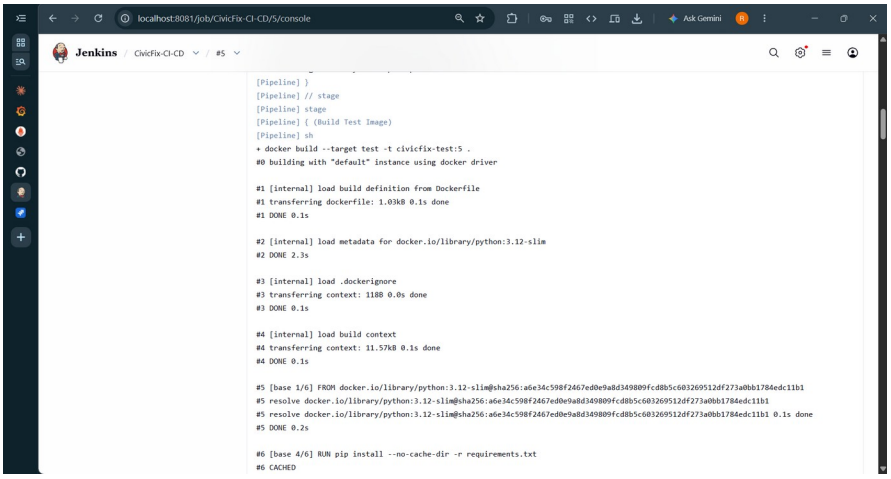
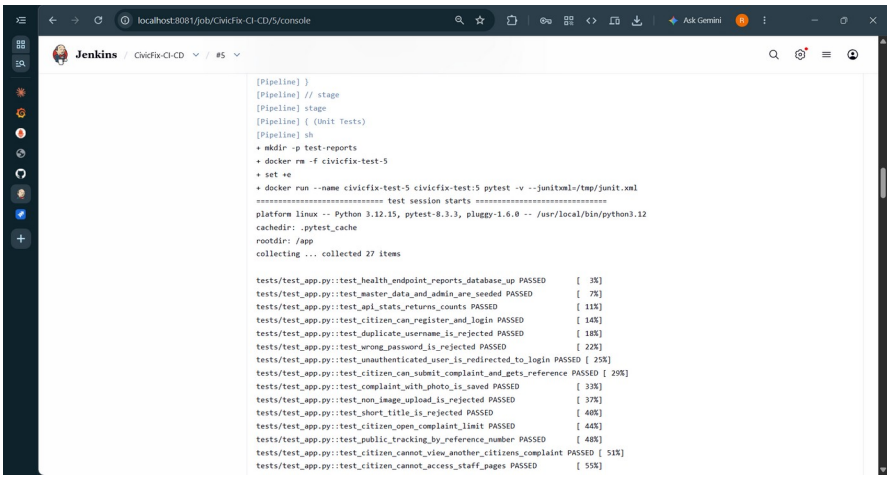
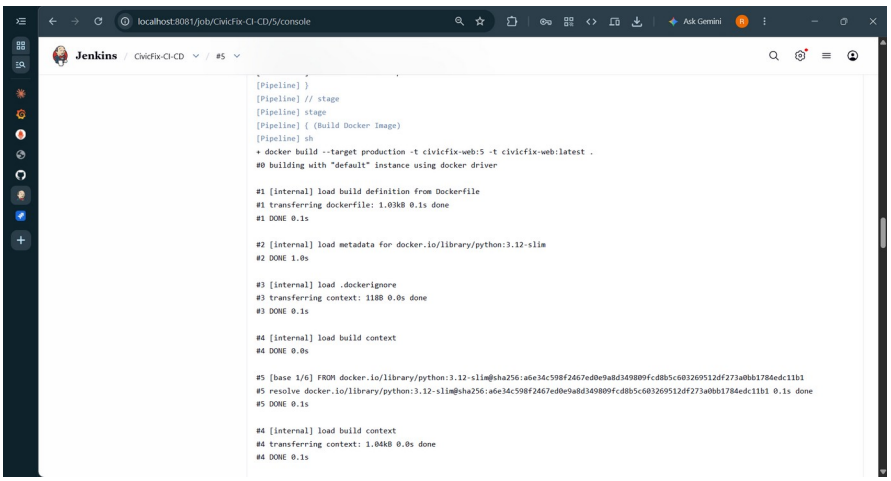


Figure 6.1: Automated testing evidence is referenced through Jenkins test results; no separate local pytest screenshot is included.


localhost:8081/job/CivicFix-CI-CD/5/console
Jenkins
CivicFix-CI-CD
#5
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Test Image)
[Pipeline] sh
+ docker build --target test -t civicfix-test:5 .
#0 building with "default" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 1.03kB 0.1s done
#1 DONE 0.1s
#2 [internal] load metadata for docker.io/library/python:3.12-slim
#2 DONE 2.3s
#3 [internal] load .dockerignore
#3 transferring context: 118B 0.0s done
#3 DONE 0.1s
#4 [internal] load build context
#4 transferring context: 11.57kB 0.1s done
#4 DONE 0.1s
#5 [base 1/6] FROM docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c603269512df273a0bb1784edc11b1
#5 resolve docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c603269512df273a0bb1784edc11b1
#5 resolve docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c603269512df273a0bb1784edc11b1 0.1s done
#5 DONE 0.2s
#6 [base 4/6] RUN pip install --no-cache-dir -r requirements.txt
#6 CACHED


localhost:8081/job/CivicFix-CI-CD/5/console
Jenkins
CivicFix-CI-CD
#5
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Unit Tests)
[Pipeline] sh
+ mkdir -p test-reports
+ docker rm -f civicfix-test-5
+ set +e
+ docker run --name civicfix-test-5 civicfix-test:5 pytest -v --junitxml=/tmp/junit.xml
============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-8.3.3, pluggy-1.6.0 -- /usr/local/bin/python3.12
cachedir: .pytest_cache
rootdir: /app
collecting ... collected 27 items
tests/test_app.py::test_health_endpoint_reports_database_up PASSED [3%]
tests/test_app.py::test_master_data_and_admin_are_seeded PASSED [7%]
tests/test_app.py::test_api_stats_returns_counts PASSED [11%]
tests/test_app.py::test_citizen_can_register_and_login PASSED [14%]
tests/test_app.py::test_duplicate_username_is_rejected PASSED [18%]
tests/test_app.py::test_wrong_password_is_rejected PASSED [22%]
tests/test_app.py::test_unauthenticated_user_is_redirected_to_login PASSED [25%]
tests/test_app.py::test_citizen_can_submit_complaint_and_gets_reference PASSED [29%]
tests/test_app.py::test_complaint_with_photo_is_saved PASSED [33%]
tests/test_app.py::test_non_image_upload_is_rejected PASSED [37%]
tests/test_app.py::test_short_title_is_rejected PASSED [40%]
tests/test_app.py::test_citizen_open_complaint_limit PASSED [44%]
tests/test_app.py::test_public_tracking_by_reference_number PASSED [48%]
tests/test_app.py::test_citizen_cannot_view_another_citizens_complaint PASSED [51%]
tests/test_app.py::test_citizen_cannot_access_staff_pages PASSED [55%]


localhost:8081/job/CivicFix-CI-CD/5/console
Jenkins
CivicFix-CI-CD
#5
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Docker Image)
[Pipeline] sh
+ docker build --target production -t civicfix-web:5 -t civicfix-web:latest .
#0 building with "default" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 1.03kB 0.1s done
#1 DONE 0.1s
#2 [internal] load metadata for docker.io/library/python:3.12-slim
#2 DONE 1.0s
#3 [internal] load .dockerignore
#3 transferring context: 118B 0.0s done
#3 DONE 0.1s
#4 [internal] load build context
#4 DONE 0.0s
#5 [base 1/6] FROM docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c603269512df273a0bb1784edc11b1
#5 resolve docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c603269512df273a0bb1784edc11b1 0.1s done
#5 DONE 0.1s
#4 [internal] load build context
#4 transferring context: 1.04kB 0.0s done
#4 DONE 0.1s

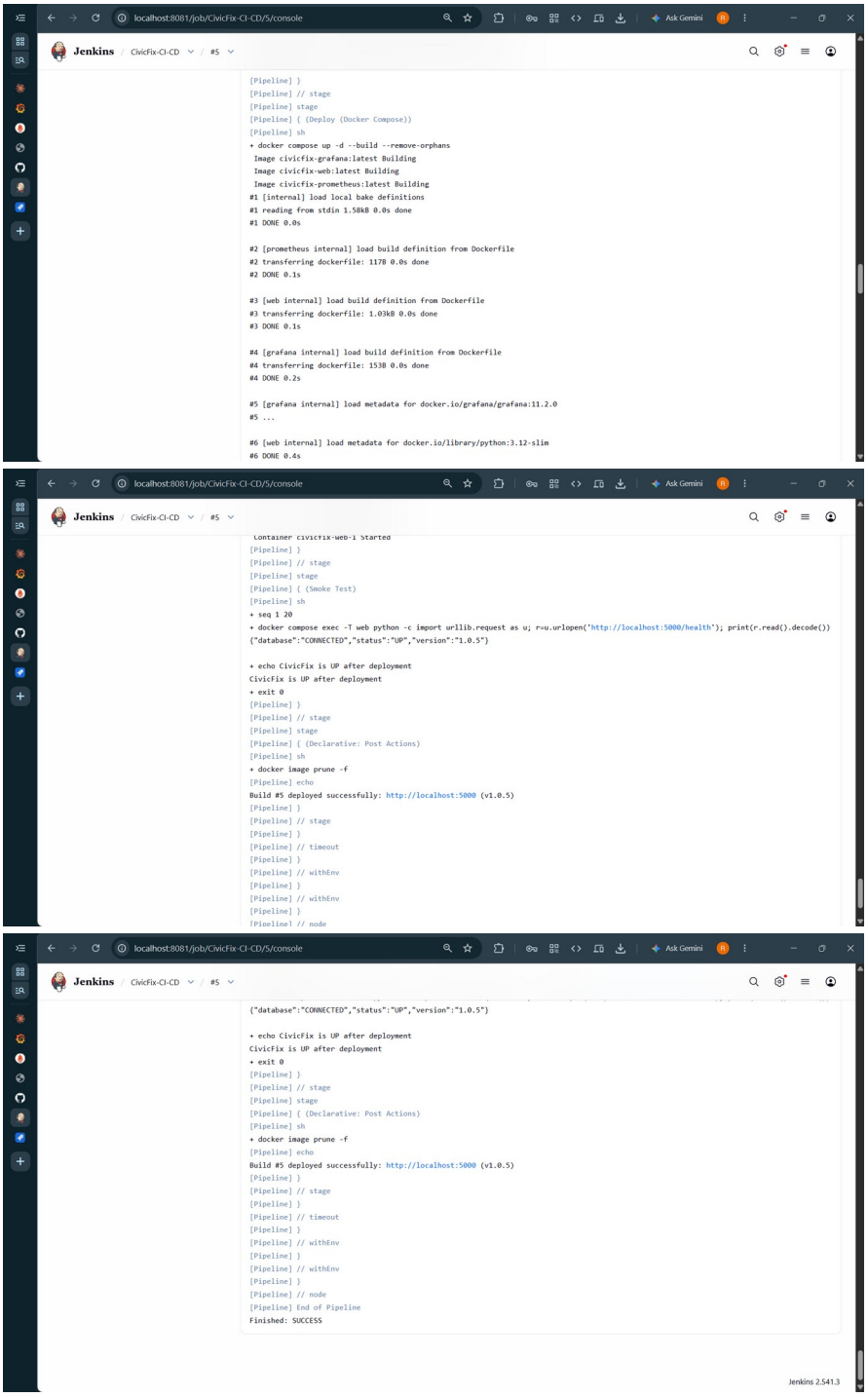


Figure 6.2: CI evidence: Jenkinsfile stage commands run from the Jenkins image

7. Implementation

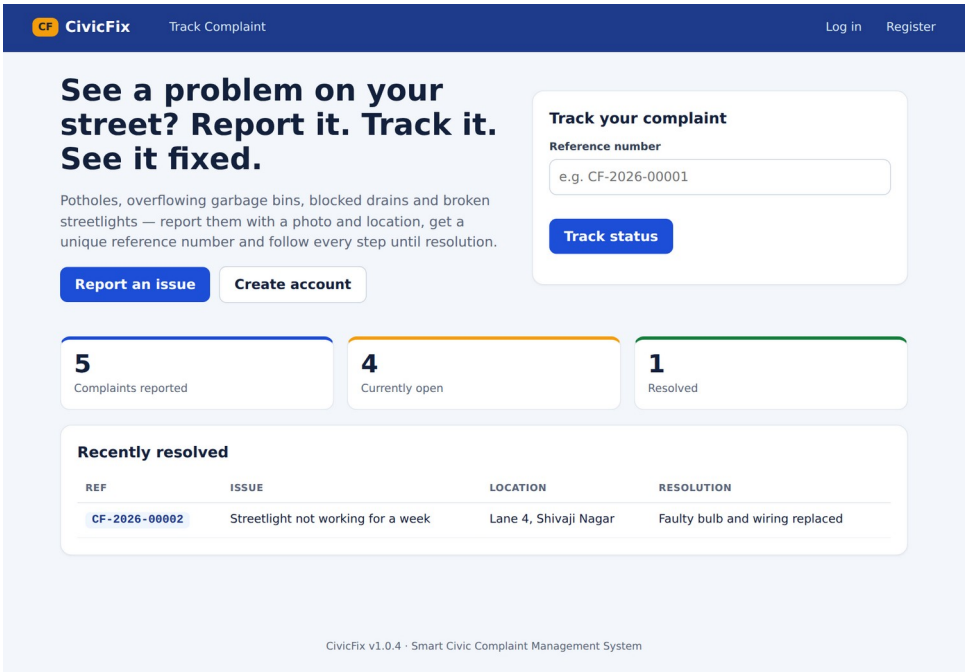


Figure 7.1: CivicFix home page with public complaint tracking

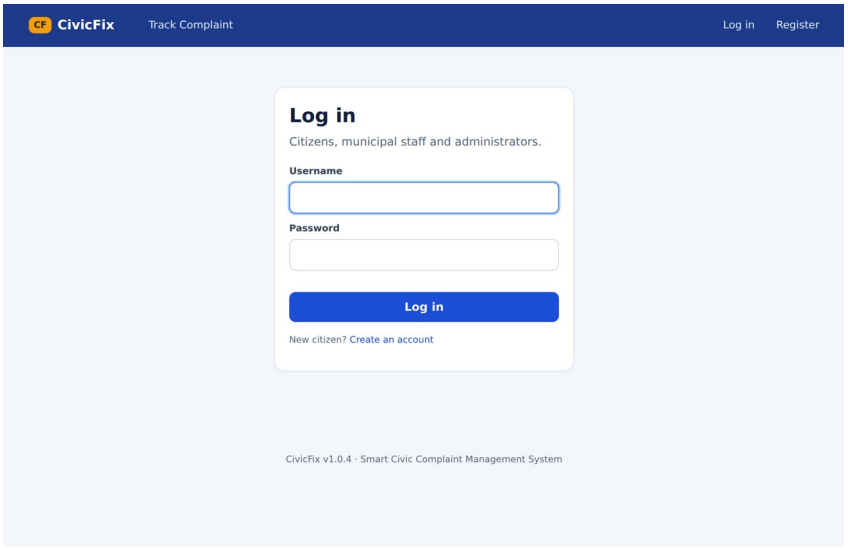


Figure 7.2: Login page

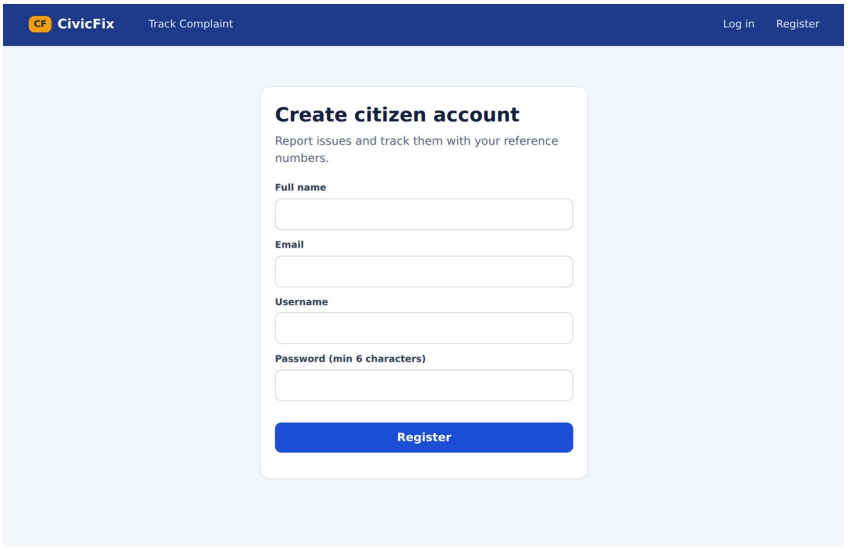


Figure 7.3: Citizen registration

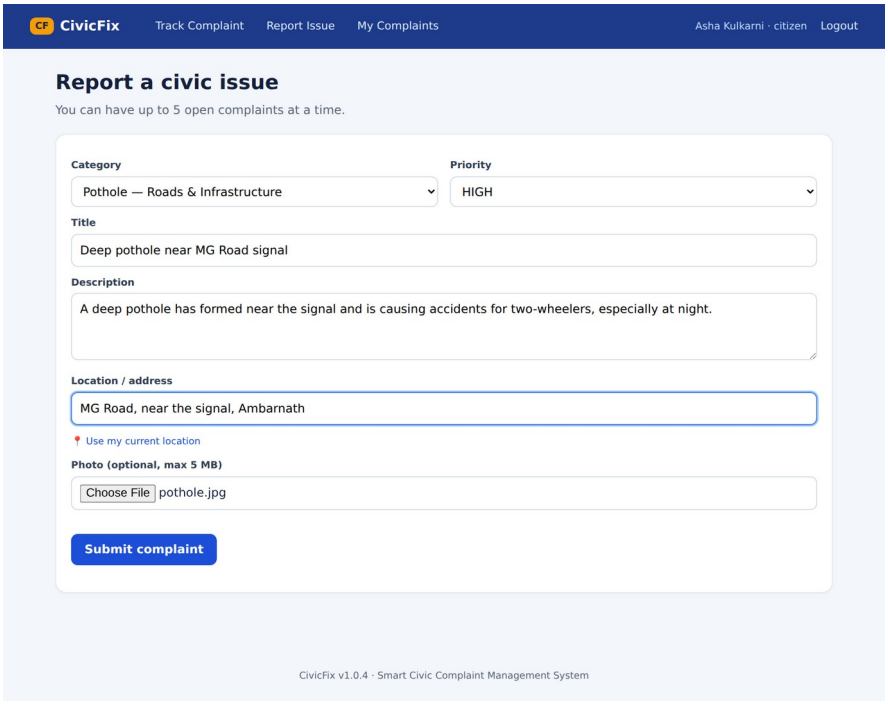


Figure 7.4: Citizen: report a civic issue with description, location and photo

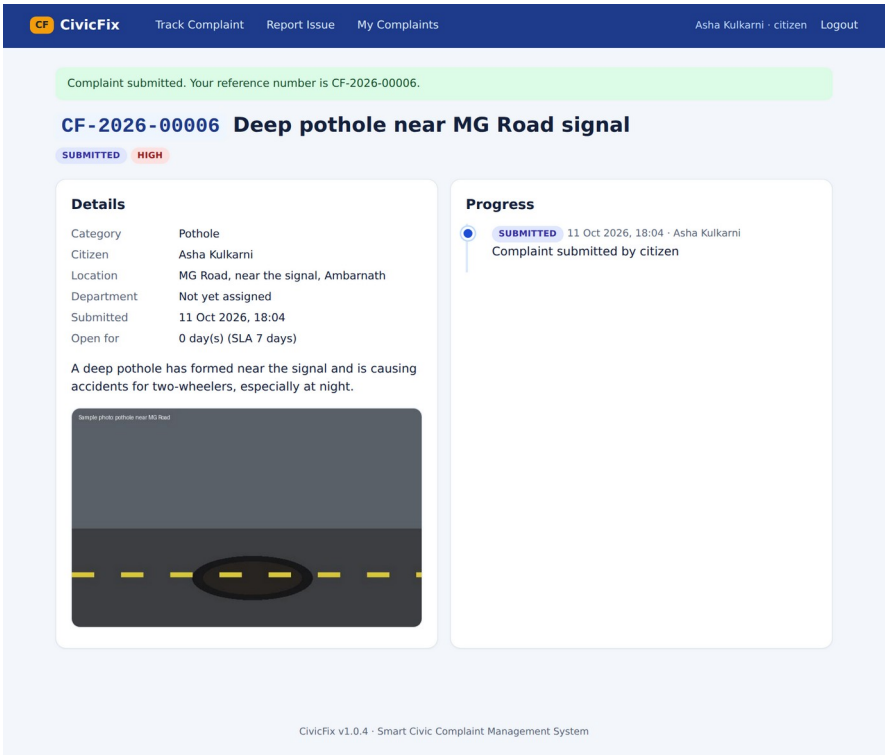


Figure 7.5: Complaint submitted — unique reference number generated, photo attached

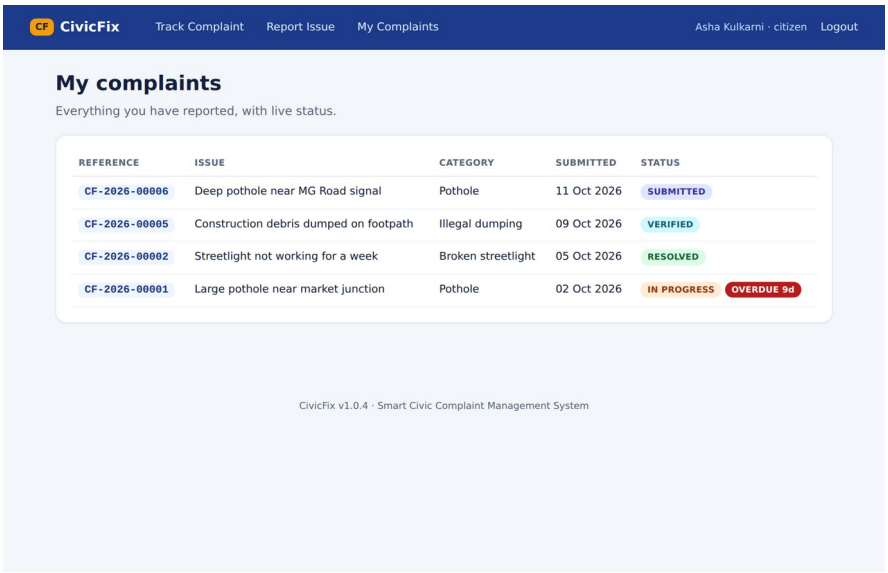


Figure 7.6: Citizen: My Complaints with statuses

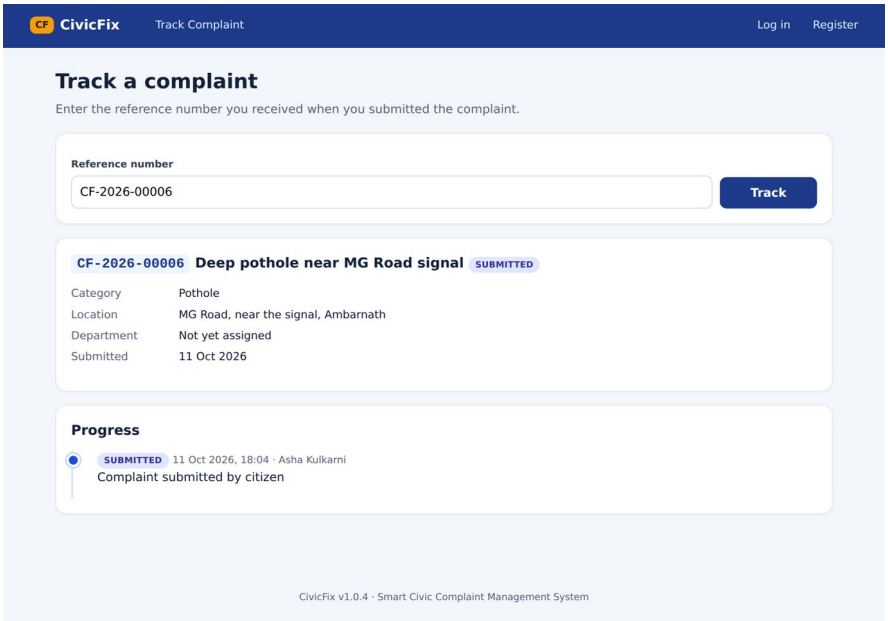


Figure 7.7: Public tracking by reference number with progress timeline

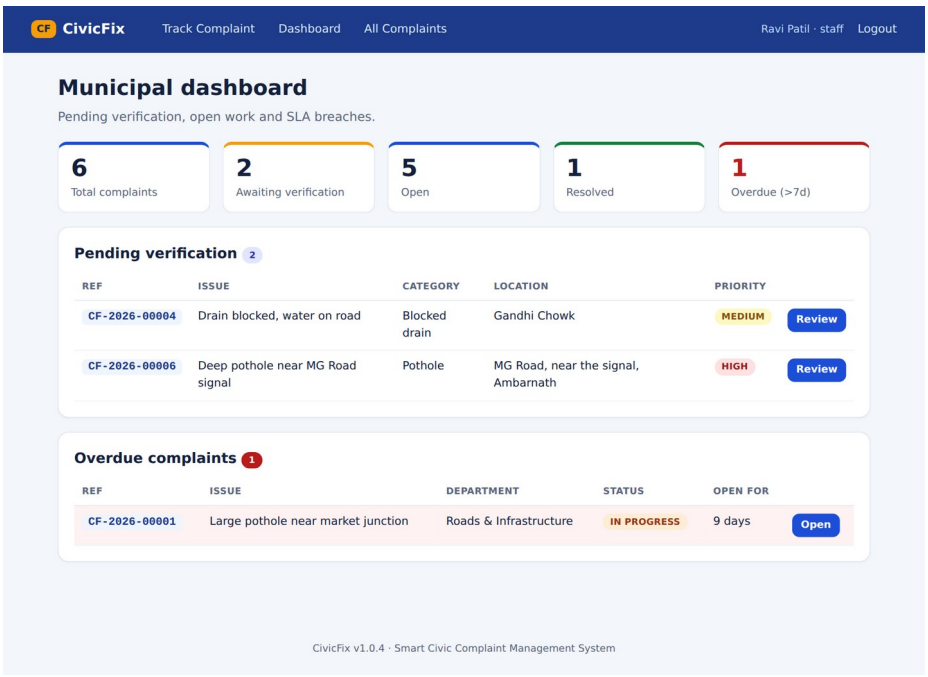


Figure 7.8: Municipal staff dashboard: pending verification, open and overdue complaints

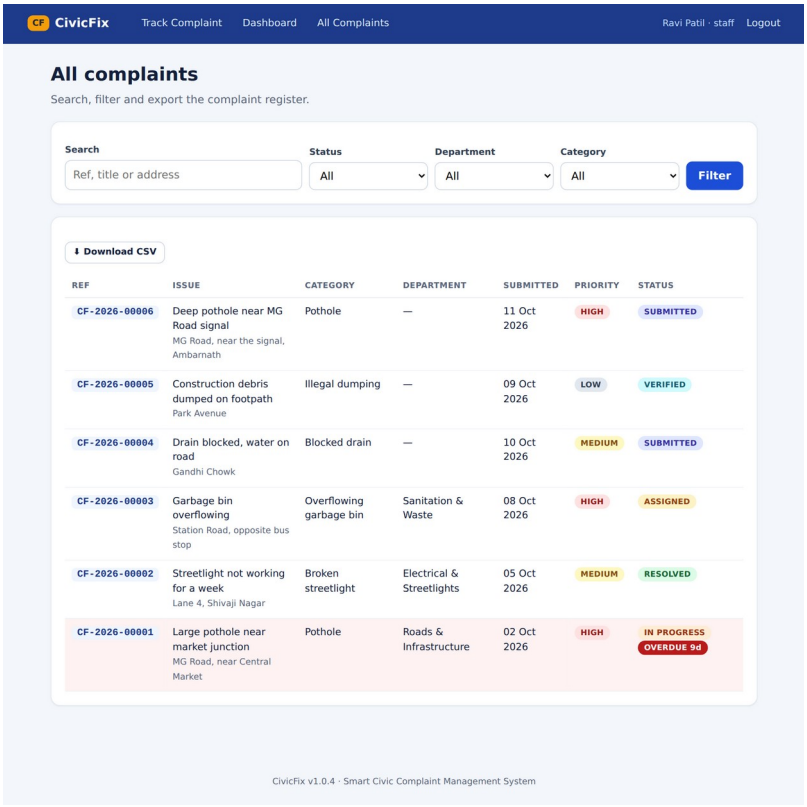


Figure 7.9: Staff: all complaints with search, filters and CSV download

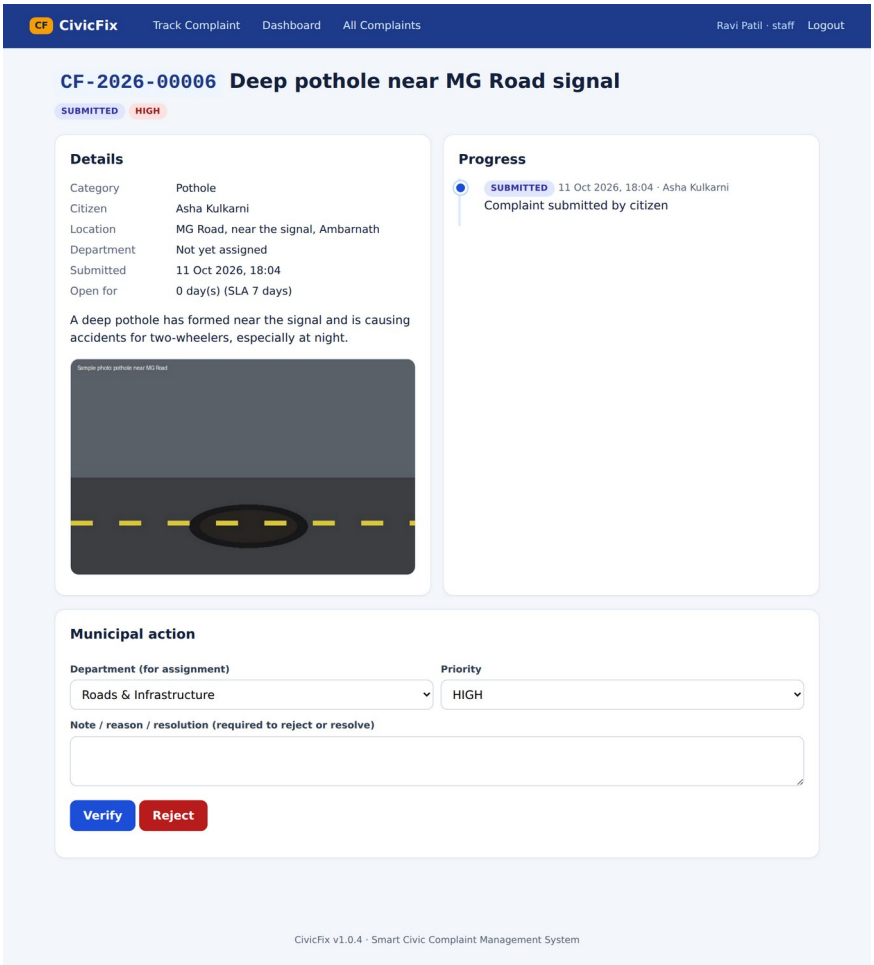


Figure 7.10: Staff: complaint details and municipal action (verify / reject / assign / resolve)

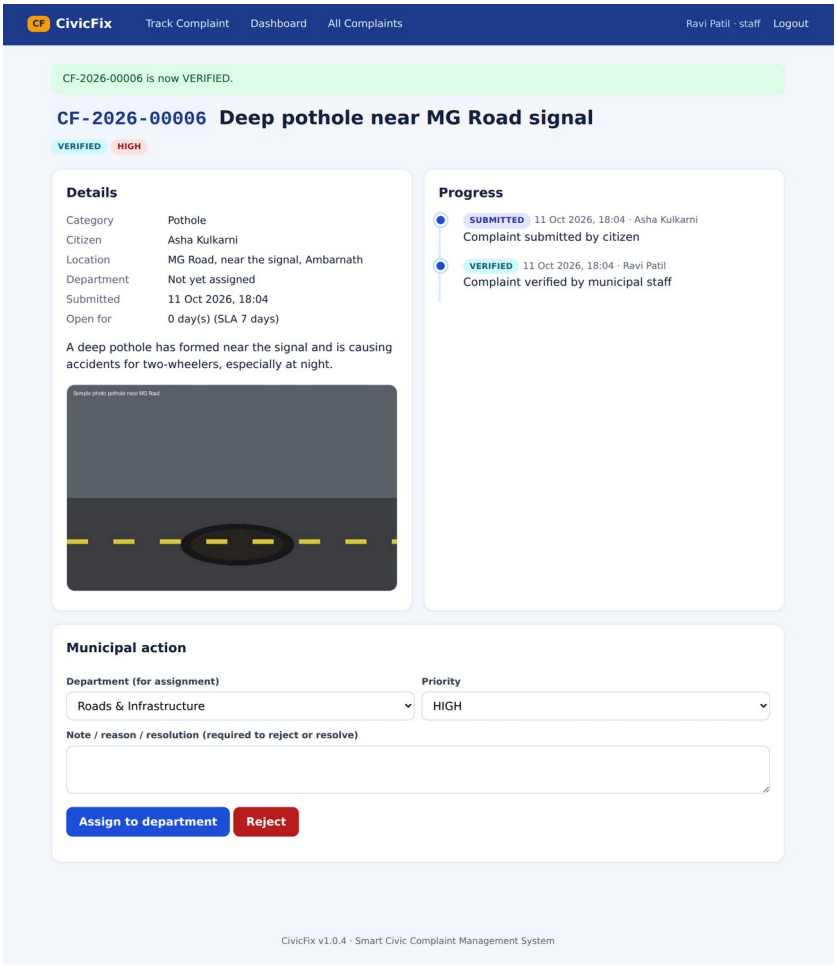


Figure 7.11: Complaint after verification — history updated

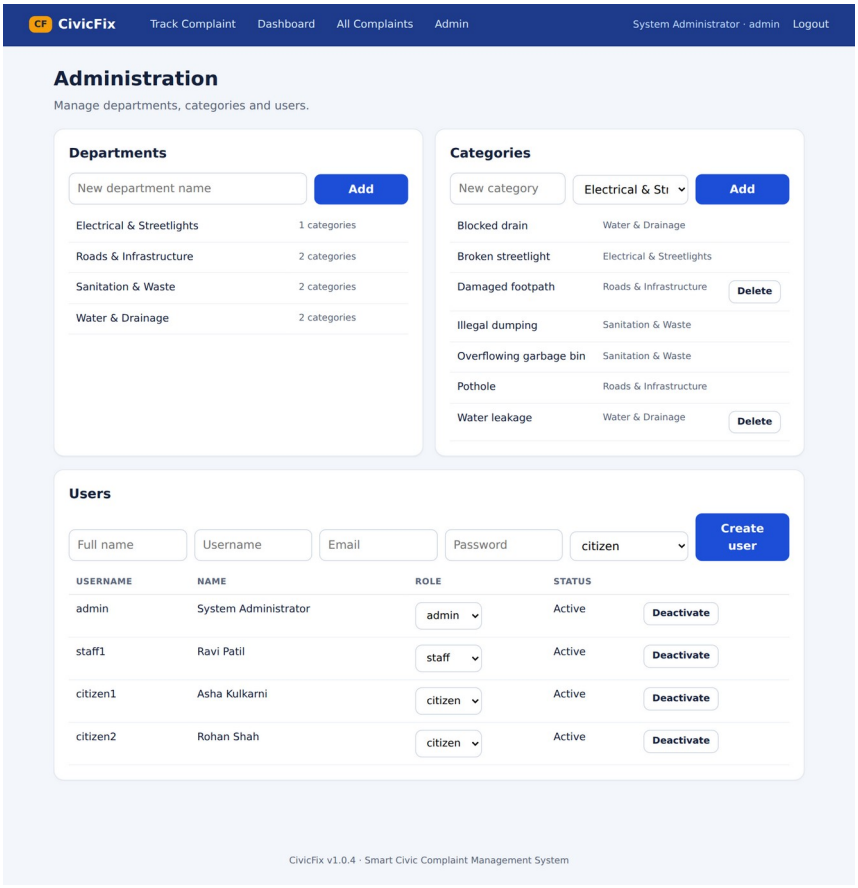


Figure 7.12: Administrator: departments, categories and users

8. Results and Discussion

8.1 Results

Area	Result
Application	A complete civic complaint system with citizen, staff and administrator roles, photo upload, unique reference numbers, public tracking, workflow history, SLA-based overdue flagging and CSV export
Version control	CivicFix source code was maintained in Git and pushed to GitHub; feature branches, a pull request and merges were used for feature development, and a faulty change was rolled back with git revert.
Containerization	The Flask application was built into a multi-stage Docker image and executed as a container with port 5000 mapped to the host; the container reported healthy.
Multi-service deployment	Docker Compose ran the Flask application, PostgreSQL, Prometheus, node-exporter and Grafana as interconnected services with persistent storage and health checking.
Continuous Integration	Jenkins was connected to GitHub through SCM polling and executed the build and pytest stages automatically after changes (27 tests passing).
Automated deployment	The Jenkins pipeline was extended to build the Docker image, deploy the application using Docker Compose and run a /health smoke test after successful testing; a failing build blocked deployment.
Agile workflow	The Scrum backlog was planned for Jira as 5 epics and 20 stories/tasks with priorities and story points (55 points), divided into three sprints, with two dependencies, and a To Do, In Progress, In Review, Done workflow.
Continuous monitoring	Prometheus collected application metrics through /metrics and server metrics through node-exporter, supported PromQL-based analysis, and Grafana visualised them.

Table 8.1: Summary of project results

8.2 Discussion

The project demonstrates that the individual laboratory experiments can be combined into one software-development lifecycle. Git and GitHub provide the source-control foundation; Docker and Docker Compose provide a consistent and multi-service execution environment; Jenkins automates verification and delivery; Jira connects development work with Agile planning; and Prometheus with Grafana extends the lifecycle into continuous monitoring. The integrated workflow reduces repetitive manual operations and provides a clear relationship between source-code changes, testing, deployment, Agile task tracking and monitoring. The failed build and the reverted build show the value of this connection in practice: a change that broke an agreed business rule was caught by the tests before deployment, and the rollback was deployed automatically. For a civic system in particular, business metrics such as overdue complaints and complaints per department make the service itself observable, not only the servers it runs on.

9. Conclusion and Future Scope

9.1 Conclusion

The development of CivicFix – Smart Civic Complaint Management System provided practical experience in applying software-development and DevOps concepts to a single integrated application. Git and GitHub provided structured version control, commit history, branching, pull requests and rollback. Docker packaged the application and its dependencies into a portable containerized environment, while Docker Compose coordinated Flask, PostgreSQL and the monitoring services as interconnected services with networking and persistent storage. Jenkins demonstrated how Continuous Integration can automatically verify changes through testing and how a CI/CD pipeline can further automate Docker image building, application deployment and a post-deployment smoke test. Jira introduced the practical application of Agile and Scrum principles by representing software features as epics, stories and tasks, planning them into sprints and moving them through stages of development, review and completion. Prometheus and Grafana demonstrated that the DevOps lifecycle does not end after deployment: application traffic, HTTP status codes, request latency, complaint counts and CPU, memory and disk usage can be observed to understand the behaviour and health of a running system.

Overall, the CivicFix mini project helped demonstrate how version control, application development, containerization, multi-service deployment, CI/CD automation, Agile project management and continuous monitoring work together as parts of a complete software-development lifecycle, while producing a useful application that makes civic issue reporting more transparent, organized and trackable.

9.2 Future Scope

Based on the technologies and workflow already demonstrated, the project can be extended with email or SMS notifications when a complaint changes status, a map view of complaints using the stored coordinates, ward-wise staff assignment, citizen feedback and rating after resolution, CSRF protection and rate limiting, Grafana or Alertmanager alerts for overdue complaints and server load, a GitHub webhook in place of polling, a separate staging environment in the pipeline, and deployment to a cloud host or Kubernetes. These are potential extensions rather than features claimed as implemented.

9.3 Learning Outcome

The project provided hands-on experience in developing, deploying, automating, managing and monitoring a web application while connecting the individual ASD & DevOps laboratory experiments into one practical workflow.

A. Appendix: Running the Project

A.1 Docker Build

```
docker build -t civicfix .
docker build --target test -t civicfix-test .    # image with pytest
```

A.2 Docker Container

```
docker run -d -p 5001:5000 --name civicfix-single civicfix
```

The container maps port 5000 to port 5001 of the host. docker images, docker ps, docker logs, docker stop and docker start were used during the implementation.

A.3 Docker Compose

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs web
```

The application runs at http://localhost:5000 (administrator admin / admin123; municipal staff staff1 / staff123; citizens citizen1 and citizen2 / citizen123), Prometheus at http://localhost:9090 and Grafana at http://localhost:3000 (admin / admin123).

A.4 Git

```
git log --oneline --graph
git remote add origin https://github.com/<your-username>/civicfix.git
git push -u origin main
git revert HEAD --no-edit
```

A.5 Monitoring

```
/metrics
civicfix_complaints_overdue
sum by (status) (rate(flask_http_request_total[1m]))
```

A.6 Jenkins

```
cd jenkins
REPO_URL=https://github.com/<your-username>/civicfix.git docker compose up -d --build
```

Jenkins opens at http://localhost:8081 (admin / admin123) with the CivicFix-CI-CD job already created.